Senior backend müsahibə bələdçisi

Doqquz mövzunun Azərbaycan dilində izahlı dərsləri. Lifecycle, DI, EF Core loading, procedure function fərqi, indekslər, fraqmentasiya, SOLID, dizayn patternləri və Task async await. Terminlər, nümunələr, müqayisələr, müsahibə sualları və cavablı məşqlər.

28 oktyabr 2026. Bu kitab qovluğun yenilənmiş tədris məzmunudur. İlkin 86 sənədin xəritəsi və düzəliş hesabatı ayrıca 00_Analiz_ve_Oxu_Plani.pdf faylındadır.

Mündəricat

Senior backend müsahibə bələdçisi 1

Lifecycle və ASP.NET Core sorğusunun ömrü 5

1 Başlamazdan əvvəl əsas anlayışlar . . . 5
2 Dörd müxtəlif ömür . . . 5
3 Sorğu addım addım haradan keçir . . . 6
4 Middleware-in əvvəl və sonra davranışı . . . 6
5 Obyektin ömrü və Dispose fərqi . . . 7
6 Tətbiqin başlanması və dayanması . . . 8
7 Yoxlama sualları və cavablar . . . 8
8 Rəsmi mənbələr . . . 8

Dependency Injection və xidmət ömürləri 9

1 Asılılıq və abstraksiya nədir . . . 9
2 IoC DI və DIP fərqi . . . 10
3 Qeydiyyat resolve və ilk yaranma . . . 10
4 Üç lifetime və dəqiq sərhədlər . . . 10
5 Captive dependency niyə yaranır . . . 11
6 Fon işində ayrıca scope . . . 11
7 Təmizlənmənin sahibi kimdir . . . 12
8 Birdən çox implementasiya və factory . . . 12
9 Səhvləri necə tanımalı . . . 12
10 Müsahibə və məşq . . . 13
11 Rəsmi mənbələr . . . 13

EF Core məlumat yükləmə növləri 14

1 Modeli başa düşək . . . 14
2 Sorğu nə vaxt işləyir . . . 14
3 Eager loading əvvəlcədən əlaqələri seçir . . . 15
4 Explicit loading sonradan açıq əmr verir . . . 15
5 Lazy loading xassəyə çatanda gizli yükləyir . . . 16
6 N+1 problemini sayla başa düşək . . . 16
7 Projection daha az məlumat seçir . . . 16
8 Single query split query və çoxalma . . . 17
9 Tracking yükləmə növü deyil . . . 17
10 Seçim və müsahibə məşqi . . . 18
11 Rəsmi mənbələr . . . 18

Stored procedure və function fərqi **19**

1 Rutin parametr və nəticə . . . 19
2 Eyni meyarlarla müqayisə . . . 19
3 Scalar function nümunəsi . . . 20
4 Cədvəl qaytaran iki function növü . . . 20
5 Procedure üç fərqli nəticə verə bilər . . . 21
6 Transaction və xəta sərhədi . . . 22
7 Performans və təhlükəsizlik mifləri . . . 22
8 Seçim məşqləri və cavablar . . . 23
9 İlkin SQL nümunələrində konkret düzəlişlər . . . 23
10 Rəsmi mənbələr . . . 24

SQL indeksləri və sütuna indeks verməyin təsiri **25**

1 Əvvəlcə cədvəl sətir səhifə və açar . . . 25
2 B plus tree necə axtarır . . . 25
3 SQL Server clustered və nonclustered . . . 26
4 Sütuna indeks verəndə nə dəyişir . . . 26
5 Composite indeksdə sıra niyə vacibdir . . . 26
6 Unique filtered və aşağı cardinality . . . 27
7 SARGability və funksiyanın təsiri . . . 27
8 PostgreSQL indeksləri ayrıca dünyadır . . . 28
9 Planı necə qiymətləndirməli . . . 29
10 Müsahibə məşqi . . . 29
11 Rəsmi mənbələr . . . 30

SQL fraqmentasiya və defraqmentasiya **31**

1 Səhifə və məntiqi sıra . . . 31
2 Xarici daxili və fayl səviyyəli problem . . . 31
3 Page split və yazmalar . . . 31
4 SQL Server-də nəyi ölçürük . . . 32
5 Reorganize nə edir . . . 32
6 Rebuild nə edir . . . 33
7 Hansı halda hansı əməliyyat . . . 33
8 PostgreSQL bloat VACUUM və REINDEX . . . 34
9 Columnstore ayrıca qiymətləndirilir . . . 34
10 Müsahibə cavabı və məşq . . . 34
11 Rəsmi mənbələr . . . 35

SOLID prinsiplərinin səbəbi və tətbiqi **36**

1 Dizaynı qiymətləndirmək üçün sözlük . . . 36
2 S Single Responsibility Principle . . . 36
3 O Open Closed Principle . . . 36
4 L Liskov Substitution Principle . . . 37
5 I Interface Segregation Principle . . . 38
6 D Dependency Inversion Principle . . . 38
7 Hamısını bir sifariş axınında birləşdirək . . . 39
8 Müsahibə sualları və məşq . . . 39
9 Mənbələr . . . 40

Dizayn patternləri və düzgün seçim **41**

1 Pattern dili və üç ailə . . . 41
2 Yaradılma patternlərinin xəritəsi . . . 41
3 Simple Factory Factory Method və Abstract Factory . . . 42
4 Builder Prototype və Singleton sərhədləri . . . 42
5 Struktur patternlərinin xəritəsi . . . 43
6 Adapter Decorator Proxy fərqi . . . 43
7 Davranış patternlərinin xəritəsi . . . 45
8 Strategy State və Template Method . . . 45
9 Mediator və Observer sərhədləri . . . 46
10 Repository Unit of Work CQRS və Outbox . . . 46
11 Seçim sualları və məşq . . . 46
12 Mənbələr və əlavə oxu . . . 47

Task async await və paralel işin məntiqi **48**

1 Proses thread concurrency parallelism . . . 48
2 Task əməliyyatın vədidir thread deyil . . . 48
3 Async metod ilk haraya qədər işləyir . . . 48
4 I/O və CPU üçün fərqli seçim . . . 49
5 Ardıcıl await və birlikdə başlatma . . . 49
6 WhenAll xətası və WhenAny . . . 50
7 CancellationToken və vaxt sərhədi . . . 51
8 Result Wait və deadlock . . . 51
9 Async void və resurs ömrü . . . 51
10 ValueTask nə vaxt lazımdır . . . 52
11 Race condition və məhdud paralellik . . . 52
12 Müsahibə cavabları və məşq . . . 53
13 Rəsmi mənbələr . . . 53

Lifecycle və ASP.NET Core sorğusunun ömrü

Lifecycle bir varlığın yaranmasından işinin bitməsinə qədər keçdiyi mərhələlərdir. .NET müsahibəsində bu söz tək bir mövzunu bildirmir. Obyektin yaddaşdakı ömrü, DI xidmətinin paylaşılma müddəti, HTTP sorğusunun emalı və tətbiqin başlama dayanma mərhələləri ayrı anlayışlardır. Bu dərs onları bir sifariş nümunəsi üzərində əlaqələndirir.

Bu sənəd `DependencyInjection.pdf`, `di_practical.pdf`, `files/02_aspnet_core.pdf` və CLR materiallarını tamamlayır. Xüsusilə “scoped hər connection üçün yaranır”, “singleton mütləq startup zamanı yaranır” və “Dispose obyektin yaddaşını silir” kimi yanlış nəticələrdən qoruyur.

1 Başlamazdan əvvəl əsas anlayışlar

Proses əməliyyat sisteminin işləyən proqram üçün ayırdığı mühitdir. Prosesin yaddaşı və resursları olur. **Thread** proses daxilində kodu icra edən axındır. Eyni prosesdə bir neçə thread eyni obyektlərə çata bilər. **Server** burada şəbəkədən gələn sorğulara cavab verən tətbiqdir. **HTTP** müştəri ilə server arasında sorğu və cavabların qaydasını müəyyən edən protokoldur. Müştəri brauzer, mobil proqram və ya başqa server ola bilər.

Request bir HTTP sorğusudur: məsələn, `GET /orders/42`. `GET` məlumat oxumaq niyyətini, `/orders/42` isə ünvanı göstərir. **Response** serverin cavabıdır. Status kodu nəticənin kateqoriyasını bildirir: `200` uğurlu, `404` tapılmadı, `500` server xətası. **Endpoint** konkret sorğunun çatdığı işləyici kod və onu seçən qaydalardır. **Instance** sinifdən yaradılmış konkret obyekt nüsxəsidir.

DI asılılıqların obyektə xaricdən verilməsidir. **Container** hansı xidmətin necə yaradılacağını bilən mexanizmdir. **Scope** xidmət nüsxələrinin paylaşılması və təmizlənməsi üçün sərhəddir. Scope bir C# `{ }` bloku ilə eyni anlayış deyil, ayrıca yaradılan DI obyektidir. HTTP sorğusu üçün framework adətən bir scope yaradır.

2 Dörd müxtəlif ömür

Ömür	Başlanğıc və son
Tətbiq	Host qurulur, xidmət etməyə başlayır, dayanma siqnalı alır və bağlanır. Host tətbiqin işləməsini idarə edən çərçivədir.
HTTP sorğusu	Server sorğunu qəbul edir, emal zəncirindən keçirir, cavabı tamamlayır.
DI xidməti	Container xidməti yaradır və lifetime qaydasına uyğun paylaşır; sahibi bitəndə resurslarını təmizləyir.
Yaddaşdakı obyekt	Yaddaş ayrılır, konstruktor işləyir, istinadlarla istifadə olunur; əlçatmaz olduqdan sonra GC yaddaşı sonradan geri ala bilər.

Bir sorğu bitə bilər, amma singleton yaşamağa davam edər. Bir obyekt `Dispose` oluna bilər, amma ona istinad qalırsa yaddaşda qalar. Eyni TCP bağlantısı ilə bir neçə HTTP sorğusu gedə bilər; buna görə scope-u bağlantı ilə eyniləşdirmə. TCP şəbəkədə baytların etibarlı çatdırılmasını təmin edən nəqliyyat protokoludur.

3 Sorğu addım addım haradan keçir

Müştəri `GET /orders/42` göndərir. Ünvanın IP-yə çevrilməsi DNS-in işidir; IP şəbəkədə cihazın ünvanıdır. HTTPS istifadə olunursa TLS bağlantının şifrələnməsini və server kimliyinin yoxlanmasını təmin edir. Bu şəbəkə mərhələləri hər sorğuda sıfırdan təkrarlanmaya bilər, çünki mövcud bağlantı təkrar istifadə oluna bilər.

Kestrel ASP.NET Core-un HTTP serveridir. Qarşısında reverse proxy ola bilər: məsələn, daxil olan trafiki tətbiqə ötürən server. Kestrel sorğu üçün `HttpContext` yaradır. Bu obyekt sorğu ünvanını, başlıqları, cavabı, istifadəçi məlumatını və ləğv siqnalını bir yerdə saxlayır. Header, yəni başlıq, sorğu və cavabla gedən əlavə məlumatdır; məsələn, məzmunun formatı.

Sonra sorğu **middleware pipeline** daxilində irəliləyir. Middleware sorğunu qəbul edən, növbəti mərhələni çağıra və cavab qayıdanda əlavə iş görə bilən komponentdir. Pipeline bu komponentlərin ardıcıllığıdır. Xəta tutma, təhlükəsiz ünvana yönləndirmə, routing, kimlik yoxlama və icazə yoxlama bu zəncirə daxil ola bilər.

Routing ünvan və HTTP metoduna əsasən endpoint seçir. **Authentication** “kimdir?” sualına cavab verir. **Authorization** “bu əməliyyatı edə bilərmi?” sualını yoxlayır. Kimliyi məlum istifadəçinin də bütün sifarişlərə giriş icazəsi olmaya bilər. `401` adətən etibarlı kimlik məlumatının tələb olunduğunu, `403` isə icazənin verilmədiyini bildirir.

MVC controller işlənirsə, model binding sorğudakı məlumatı metod parametrlərinə çevirir. Validation həmin məlumatın qaydalara uyğunluğunu yoxlayır. Controller HTTP ilə iş məntiqi arasında əlaqə qurur. Action onun konkret sorğunu işləyən metodudur. Filter isə MVC action və nəticə emalının müəyyən mərhələlərinə qoşulan mexanizmdir. Minimal API istifadə olunursa ayrıca controller və MVC filter mərhələsi olmaya bilər.

Controller xidmət çağırır, xidmət lazım gəlsə verilənlər bazası ilə işləyir. **DbContext** EF Core-da sorğuların göndərilməsi və obyekt dəyişikliklərinin izlənməsi üçün istifadə olunan iş vahididir. EF Core C# obyektlərini cədvəl məlumatları ilə əlaqələndirən kitabxanadır. Nəticə DTO-ya çevrilə bilər: DTO şəbəkə ilə göndərmək üçün seçilmiş məlumatları saxlayan sadə obyektdir. Serialization obyektin JSON kimi ötürülən formata çevrilməsidir. JSON açar və dəyərlərlə mətn şəklində məlumat formatıdır.

Nəhayət, cavab geri qayıdır və scope təmizlənir. Sorğu ləğv ediləndə `RequestAborted` token-i bunu bildirir. Token əməliyyata “dayanmaq istənilir” siqnalını ötürür; kod və sürücü onu nəzərə almalıdır. Cavab yazılmağa başladıqdan sonra status və başlıqların dəyişdirilməsi məhdudlaşır.

4 Middleware-in əvvəl və sonra davranışı

Aşağıdakı C# fraqmenti `Program.cs` daxilində `app.Run()` çağırışından əvvəl yerləşdirilir. `app` qurulmuş `WebApplication` obyektidir. `Console.WriteLine` ekrana qeyd yazır. Lambda `=>` işarəsi ilə verilən qısa funksiya ifadəsidir.

C#

```
app.Use(async (context, next) =>
{
    Console.WriteLine("A evvel");
    await next(context);
    Console.WriteLine("A sonra");
});

app.Use(async (context, next) =>
{
    Console.WriteLine("B evvel");
    await next(context);
    Console.WriteLine("B sonra");
});

app.MapGet("/hello", () => "Salam");
```

Uğurlu `/hello` sorğusunda sıra belədir: A əvvəl, B əvvəl, endpoint, B sonra, A sonra. `next` növbəti komponenti təmsil edir. `await` həmin asinxron əməliyyatın tamamlanmasını bloklamadan gözləyir. Növbəti hissə xəta atarsa “sonra” sətri avtomatik işləməyə bilər; mütləq təmizlik üçün `try/finally` lazımdır. `finally` nəticədən asılı olmayaraq icra edilən təmizlik blokudur.

Middleware `next` çağırmadan cavab qaytarırsa zənciri qısaldır. Məsələn, icazəsiz sorğuya cavab verən mərhələ bazaya müraciəti dayandıra bilər. Sıra buna görə əhəmiyyətlidir. Xəta tutan komponent erkən, authentication isə authorization-dan əvvəl yerləşdirilir. Konkret sıra istifadə olunan funksiyalara görə qurulur; bütün tətbiqlər üçün eyni kor siyahı tətbiq edilmir.

5 Obyektin ömrü və Dispose fərqi

Konstruktor obyekt yaradılarkən onun ilkin vəziyyətini quran metoddur. **İstinad** obyektə çatmağa imkan verən əlaqədir. **Managed heap** .NET-in obyektlər üçün idarə etdiyi yaddaş sahəsidir. **GC**, Garbage Collector, proqramın artıq çata bilmədiyi obyektlərin yaddaşını avtomatik geri alan mexanizmdir. Obyektin faydalı işi bitən kimi GC-nin işləməsi zəmanətli deyil.

Fayl və şəbəkə bağlantısı kimi resurslar vaxtında buraxılmalıdır. **IDisposable** bunun üçün `Dispose()` müqaviləsini, **IAsyncDisposable** isə asinxron `DisposeAsync()` müqaviləsini verir. `using` və `await using` müvafiq təmizlik metodunun çağırılmasını təşkil edir. Dispose yaddaşı dərhal boşaltmır və GC-ni məcbur etmir.

C#

```
using var file = File.OpenRead("data.txt");
// file-dan oxuma aparilir.
// Cari blok bitende Dispose cagrilir.
```

Bu fraqment üçün `System.IO` ad sahəsi lazımdır. Ad sahəsi əlaqəli tipləri qruplaşdıran addır. `var` dəyişənin tipinin sağdakı ifadədən kompilyator tərəfindən müəyyən edilməsidir. Kompilyator C# mətnini icra edilən formaya çevirən alətdir. `//` kodda işlədilməyən izah sətridir.

Container yaratdığı və təmizlənməsini izlədiyi disposable xidmətləri scope və ya provider bitəndə təmizləyir. Provider container-dən xidmət nüsxələrini əldə etməyə imkan verən obyektdir. Özün yaratdığın obyekti `AddSingleton(existingObject)` ilə verəndə onun təmizlənməsi sənin məsuliyyətindir. DI-dan aldığın `DbContext`-i onu alan hər servis ayrıca dispose etməməlidir; sahibi olan scope bunu etməlidir.

6 Tətbiqin başlanması və dayanması

Başlanğıcda konfiqurasiya oxunur, xidmətlər qeydiyyatdan keçir, host qurulur və server sorğuları dinləməyə başlayır. Konfiqurasiya bağlantı ünvanları kimi davranış parametrləridir. Qeydiyyat sinfin dərhal yaradılması demək deyil: singleton çox vaxt ilk dəfə tələb olunanda yaradılır.

Hosted service host tərəfindən başladılan və dayandırılan işdir. `BackgroundService` uzunmüddətli fon işi üçün baza sinifdir və adətən singleton kimi idarə edilir. Fon işində hər əməliyyat üçün ayrıca scope yaratmaq olar. Dayanma token-i gəldikdə dövr dayanmalı, resurslar buraxılmalıdır. **Graceful shutdown** davam edən işlərə məhdud vaxt ərzində səliqəli bitmək imkanı verən dayanmadır. Proses qəfil öldürülsə `finally` və Dispose-un işləməsi zəmanətli deyil; vacib işlər davamlı yaddaşda saxlanmalıdır.

HTTP sorğusunun içində başladılıb gözlənilməyən fon işi sorğunun scope-undan daha uzun yaşaya bilər. İş həmin `DbContext`-dən istifadə etdikdə artıq dispose olunmuş obyektlə qarşılaşar. Etibarlı fon emalı üçün ayrıca scope və lazım olduqda davamlı növbə istifadə edilir. Növbə işləri sonradan emal olunmaq üçün saxlayan mexanizmdir.

7 Yoxlama sualları və cavablar

Bir istifadəçi iki sorğu göndərsə scoped obyekt eynidirmi? Adətən xeyr. Scope istifadəçinin hesabına bağlı deyil, ayrı HTTP sorğularına bağlıdır.

Eyni sorğuda iki thread işləsə iki DbContext yaranırmı? Scope avtomatik thread-ə görə bölünmür. Eyni scoped context paylaşıla bilər və paralel istifadə təhlükəlidir. DbContext eyni anda bir neçə əməliyyatı təhlükəsiz icra etmək üçün hazırlanmayıb.

Dispose çağırılıbsa obyekt yox olurmu? Xeyr. Resurslar buraxılır, amma obyektə istinad qala bilər. Onu yenidən istifadə etmək xəta və ya yanlış davranış yarada bilər.

Singleton bütün serverlərdə birdirmi? Xeyr. Eyni qeydiyyat üçün bir root provider daxilində paylaşılır. İki prosesin və ya iki server nüsxəsinin ayrı singleton-ları var.

Məşq. A və B middleware-lərini əlavə et. B-də `next` çağırmadan cavab yaz. A-nın “sonra” hissəsi işləyəcək, endpoint işləməyəcək. Sonra B-də xəta yarat və A-nın `try/finally` ilə təmizlik etdiyini müşahidə et. Məqsəd giriş sırası ilə geri qayıtma sırasını ayırmaqdır.

8 Rəsmi mənbələr

- Microsoft Learn - ASP.NET Core middleware
- Microsoft Learn - DI service lifetimes
- Microsoft Learn - DbContext lifetime

Mənbələr 28 oktyabr 2026 tarixində yoxlanılıb. Kod hissələri tədris fraqmentləridir; tam layihə və bütün `using` sətirləri göstərilmir.

Dependency Injection və xidmət ömürləri

Dependency Injection, qısa adı DI, bir obyektin ehtiyac duyduğu başqa obyektləri kənardan almasıdır. Bu dərsdə “nə qeydiyyatdan keçirirəm?”, “obyekt nə vaxt yaranır?”, “kim paylaşır?” və “kim təmizləyir?” suallarını ayrı cavablandıracağıq. Əsas məqsəd sintaksisi əzbərləməkdən əlavə yanlış lifetime seçiminin səbəb olduğu xətanı qabaqcadan görməkdir.

İlkin materiallar: `DependencyInjection.pdf`, `di_practical.pdf`, `files/03_di_architecture.pdf`. Həmin sənədlərdəki scope, yaranma anı, DIP və xidmətlərin təmizlənməsi izahlarını burada dəqiqləşdiririk.

1 Asılılıq və abstraksiya nədir

`OrderService` sifarişi saxlamalıdırsa və bu iş üçün `OrderStore` istifadə edirsə, store onun **dependency**, yəni asılılığıdır. **Sinif** məlumat və davranışı təsvir edən tipdir. **Obyekt** həmin tipdən yaradılmış nüsxədir. **Interface** sinfin hansı əməliyyatları təqdim etməli olduğunu göstərən müqavilədir. **Implementation** müqavilənin konkret necə yerinə yetirildiyidir.

Abstraksiya istifadəçiyə vacib davranışı göstərib daxili təfərrüatı gizlədən modeldir. Interface abstraksiyanın bir üsuludur, yeganə üsulu deyil. **Coupling** bir hissənin digər hissənin təfərrüatına nə qədər bağlı olmasıdır. Concrete SQL sinfinə birbaşa bağlı xidmət onu dəyişəndə daha çox təsirlənir. Müqaviləyə bağlı xidmət eyni davranış müqaviləsini qoruyan başqa implementasiya ilə işləyə bilər.

C#

```
public interface IOrderStore
{
    void Save(string order);
}

public sealed class OrderService
{
    private readonly IOrderStore _store;

    public OrderService(IOrderStore store)
    {
        _store = store;
    }

    public void Place(string order)
    {
        _store.Save(order);
    }
}
```

`public` üzvün kənardan görünməsini, `private` yalnız sinif daxilində görünməsini bildirir. `sealed` sinifdən miras almağı bağlayır. `readonly` həmin sahənin istinadının konstruktor xaricində dəyişdirilməsini qadağan edir; göstərdiyi obyektin daxilini avtomatik dəyişməz etmir. `void` nəticə dəyəri qaytarılmadığını, `string` mətn tipini bildirir. `Place` iş metodudur, `OrderService(...)` konstruktordur.

Bu sinif store-u özü yaratmır. `new OrderService(new MemoryOrderStore())` yazmaq da manual DI-dır. `MemoryOrderStore` müqaviləni yerinə yetirən, yaddaşda saxlayan nümunə sinif ola bilər. Container DI üçün məcburi deyil. Eyni zamanda DI mütləq interface tələb etmir; konkret sinif də xaricdən verilə bilər.

2 IoC DI və DIP fərqi

IoC, Inversion of Control, idarəetmənin çərçivəyə və ya başqa mexanizmə ötürülməsidir. Məsələn, HTTP gələndə hansı controller-i çağıracağını framework seçir. DI obyekt asılılıqlarının verilməsi sahəsində IoC tətbiqidir.

DIP, Dependency Inversion Principle, yüksək səviyyəli iş qaydalarının aşağı səviyyəli texniki detallara deyil, uyğun abstraksiyalara bağlı olmasını istəyən dizayn prinsipidir. “Controller yalnız service-i bilir” qat qaydası təkbaşına DIP deyil. Service birbaşa konkret SQL sinfinə bağlıdırsa qatlar olsa da DIP məqsədi tam əldə olunmaya bilər. DI isə həmin dizayndakı asılılığın xaricdən verilmə texnikasıdır. DI ilə zəif dizaynı da qurmaq mümkündür.

Composition root tətbiqin konkret implementasiyaları birləşdirdiyi mərkəzdir; .NET-də çox vaxt `Program.cs` və qeydiyyat funksiyalarıdır. Biznes metodlarının hər yerində container axtarmaq əvəzinə obyekt qrafı burada qurulur. Obyekt qrafı bir obyektin digər obyektlərdən ibarət asılılıq şəbəkəsidir.

3 Qeydiyyat resolve və ilk yaranma

`IServiceCollection` xidmət təsvirlərinin siyahısıdır. Qeydiyyat “bu tip istənilsə bunu yarat və bu ömürlə paylaş” qaydasını əlavə edir. **Resolve** həmin qaydaya əsasən xidmət nüsxəsini əldə etməkdir. `IServiceProvider` resolve edən obyektdir. `GetRequiredService<T>()` qeydiyyat yoxdursa xəta verir; `GetService<T>()` isə uyğun hallarda `null`, yəni obyektin olmaması dəyərini qaytarır.

C#

```
builder.Services.AddScoped<IOrderStore, SqlOrderStore>();
builder.Services.AddScoped<OrderService>();
```

Bu fraqmentdə `SqlOrderStore` ayrıca yazılmış implementasiyadır, `builder` tətbiq qurucusudur. `OrderService` tələb olunanda container konstruktorundakı `IOrderStore` üçün də resolve edir. **Generic** `<T>` yazılışı müxtəlif tiplərlə işləyən ümumi tip və ya metod mexanizmidir. `<IOrderStore, SqlOrderStore>` müqavilə ilə implementasiyanı bağlayır.

Container lazım olmayan bütün servisləri startup-da yaratmır. Singleton çox vaxt ilk resolve zamanı yaranır. `AddSingleton(new Cache())` kimi hazır nüsxə verilirsə nüsxəni proqramçı artıq yaradıb. Qeydiyyatın mövcudluğu ilə constructor-un işə düşməsi ayrı hadisələrdir.

4 Üç lifetime və dəqiq sərhədlər

Lifetime	Paylaşılma qaydası
Transient	Container hər resolve zamanı yeni nüsxə yaradır. Eyni nüsxəni dəyişəndə saxlayıb yenidən istifadə etsən yeni obyekt yaranmır.
Scoped	Eyni scope və eyni qeydiyyat daxilində eyni nüsxə qaytarılır. Başqa scope ayrıca nüsxə alır.
Singleton	Eyni root provider və qeydiyyat daxilində bir nüsxə paylaşılır. Bir neçə proses və ya ayrıca provider ayrıca nüsxə saxlayır.

İki resolve və iki scope düşün: S1 daxilində T1 və T2 transient nüsxələr, C1 və C1 scoped nüsxələr, G və G singleton nüsxələr alınır. S2 daxilində T3 və T4, C2 və C2, yenə G və G alınır. Beləcə transient üçün resolve sayı, scoped üçün scope sayı, singleton üçün provider sərhədi əsasdır.

State obyektin saxladığı dəyişə bilən məlumatdır. **Stateless** xidmət çağırışlar arasında istifadəçiyə aid vəziyyət saxlamır. Stateless olması xidməti avtomatik singleton etməyə məcbur etmir. **Thread safety** eyni obyektə paralel girişlərin düzgün davranışı pozmamasıdır. Singleton-da request-ə aid istifadəçi ID-si saxlamaq iki istifadəçinin məlumatını qarışdıra bilər. Sadəcə onu singleton qeydiyyata almaq thread safety yaratmır.

5 Captive dependency niyə yaranır

Captive dependency uzunömürlü obyektin daha qısa ömür üçün nəzərdə tutulan obyekti özündə saxlamasıdır. Singleton constructor-u scoped `DbContext` alıb sahədə saxlayarsa context sorğu bitəndə yox olmalı olduğu sərhəddən uzun istifadə edilər. Scope validation aktivdirsə bu səhv erkən aşkar edilir. Validation qaydaların yoxlanmasıdır.

C#

```
// Problemlidir: Cache uzun, db ise qisa omurludur.
public class Cache(AppDbContext db)
{
    private readonly AppDbContext _db = db;
}
```

Burada constructor-un qısa forması C# 12 primary constructor yazılışıdır. `AppDbContext` sənin DbContext-dən törəyən sinfindir. DbContext dəyişiklik izləməsi və verilənlər bazası əməliyyatları aparır, paralel əməliyyatlara uyğun deyil. Daha uzun ömür köhnə izlənən məlumatı, yaddaş artımını və eyni anda istifadə xətalarını da yarada bilər.

Singleton-un transient alması container baxımından mümkündür. Amma singleton transient-i bir dəfə alıb saxladıqda həmin konkret obyekt də singleton qədər istifadə oluna bilər. “Transient yazmışam, deməli hər metod çağırışında yenidir” nəticəsi yanlışdır.

6 Fon işində ayrıca scope

Aşağıdakı nümunədə singleton işçi yalnız scope fabriki saxlayır. Hər `CountAsync` çağırışı yeni scope açır, onun context-i ilə işi bitirir və scope-u bağlayır. Scope-dan alınmış context sahədə saxlanmır.

C#

```
public sealed class Counter(IServiceScopeFactory scopes)
{
    public async Task<int> CountAsync(CancellationToken ct)
    {
        await using var scope = scopes.CreateAsyncScope();
        var db = scope.ServiceProvider
            .GetRequiredService<AppDbContext>();

        return await db.Orders.CountAsync(ct);
    }
}
```

`IServiceScopeFactory` scope yaradan framework xidmətidir. `await using` scope-un asinxron təmizlənməsini gözləyir. `Task<int>` tamamlananda tam ədəd nəticəsi verən əməliyyatı təmsil edir. `CancellationToken` ləğv siqnalıdır. `CountAsync` SQL ilə say hesablayan EF Core metodudur. Bu fraqment `Microsoft.Extensions.DependencyInjection` və `Microsoft.EntityFrameworkCore` ad sahələrindən istifadə edir. Alternativ olaraq `IDbContextFactory<T>` ilə ayrıca context yaradıla bilər; həmin yaradılmış context-i istifadə edən kod dispose etməlidir.

Adi constructor əsasında middleware tətbiq üçün bir dəfə qurula bilər. Scoped xidməti orada constructor ilə tutmaq əvəzinə `InvokeAsync` parametrindən almaq və ya hər sorğu üçün aktivləşən `IMiddleware` modelini seçmək lazımdır. `InvokeAsync` middleware-in sorğunu işləyən metodudur.

7 Təmizlənmənin sahibi kimdir

Ownership, sahiblik, resursu kimin təmizləməli olduğunu bildirir. Container yaratdığı disposable transient və scoped obyektləri scope-un, singleton-u isə provider-in sonunda təmizləyir. Disposable `IDisposable` və ya `IAsyncDisposable` ilə resurs buraxa bilən obyektdir. Constructor-la verilmiş asılılığı onu alan hər sinif ayrıca dispose etsə başqa istifadəçi də işləməyən obyekt alar.

Transient disposable obyekti root provider-dən davamlı resolve etmək container bağlanana qədər çox nüsxənin saxlanmasına səbəb ola bilər. Buna görə transient-in qeydiyyat adı həmişə qısa real yaddaş ömrünə zəmanət vermir. Root provider əsas container-dir; ayrıca yaradılmış scope deyil.

`AddSingleton(existingObject)` ilə xaricdə yaradılmış nüsxə verilibsə onun təmizlənməsini proqramçı idarə edir. `Dispose` yalnız resurs buraxır, obyektin managed yaddaşını dərhal silmir. Əlçatmaz obyektin yaddaşını GC sonradan geri ala bilər.

8 Birdən çox implementasiya və factory

Eyni interface üçün bir neçə qeydiyyat varsa, birbaşa tək xidmət resolve edərkən built-in container adətən son qeydiyyatı qaytarır. `IEnumerable<T>` isə bütün uyğun qeydiyyatları qeydiyyat sırası ilə verir. IEnumerable elementləri bir-bir gəzməyə imkan verən müqavilədir.

Factory obyektin yaradılmasını və ya uyğun nüsxənin seçilməsini ayrı yerə toplayan üsuldur. Runtime, proqram işləyən vaxt, gələn “card” və ya “cash” seçiminə görə fərqli xidmət seçilə bilər. DI-da factory delegate, çoxsaylı qeydiyyat və keyed services kimi imkanlar var. “DI runtime seçim edə bilmir” iddiası düzgün deyil. Keyed service xidmətə tipdən əlavə açar ilə müraciət etməyə imkan verir; istifadə olunan .NET versiyasında dəstəyini yoxlamaq lazımdır.

Constructor injection məcburi asılılıqları obyekt yaradılarkən verir. **Method injection** asılılığı konkret metod çağırışında verir. **Property injection** asılılığı property, yəni obyektin xassəsi ilə verir; built-in container bunu ümumi avtomatik davranış kimi təqdim etmir. Məcburi asılılığı sonradan verilə bilən property-də saxlamaq yarımçıq obyekt riskidir.

9 Səhvləri necə tanımalı

Service Locator iş metodunun container-dən gizli şəkildə xidmət axtarmasıdır. Asılılıqlar constructor-da görünmür və test çətinləşir. Ancaq composition root və scope açan infrastruktur kodunda provider-dən resolve etmək normaldır. `IServiceProvider` sözünün görünməsi təkbaşına bütün hallarda pis dizayn demək deyil.

Circular dependency A-nın B-yə, B-nin də A-ya birbaşa və ya aralıq obyektlərlə ehtiyacı olmasıdır. Həll üçün məsuliyyətləri yenidən bölmək, ortaq davranışı çıxarmaq və ya qarşılıqlı çağırışı dəyişmək lazımdır. Mediator əlavə etmək hər dövrəni avtomatik aradan qaldırmır; işləmə vaxtında yenə A-B-A çağırış dövrəsi qala bilər.

Constructor-da şəbəkə və baza çağırışı obyekt yaradılmasını gözlənilmədən bahalı və xətaya açıq edir. Constructor ilkin vəziyyəti qursun, I/O, yəni giriş çıxış əməliyyatı, ayrıca gözlənilə bilən metodda aparılsın. Biznes kodunda `BuildServiceProvider()` çağırıb ikinci container yaratmaq da singleton-ların təkrarlanmasına və lifetime qarışıqlığına səbəb ola bilər.

10 Müsahibə və məşq

Qısa cavab. DI asılılığı xaricdən verməklə obyektin yaradılma məntiqini iş məntiqindən ayırır. Container bunu avtomatlaşdırır. Transient hər resolve-da, scoped hər scope-da yenidir; singleton provider daxilində paylaşılır. Seçim paylaşılmış vəziyyətə, thread safety-yə, resurs sahibinə və əməliyyat sərhədinə əsaslanır.

Test nəyi sübut edir? Fake store saxlanılan sifarişi yaddaşda qeyd edən sadə implementasiya ola bilər. Onu `OrderService`-ə verib `Place` çağırırsan və düzgün dəyərin store-a ötürüldüyünü yoxlayırsan. Fake sadələşdirilmiş işlək əvəzləyicidir, mock isə çağırış gözləntiləri yoxlanan əvəzləyicidir. Bu test real SQL-in işlədiyini sübut etmir; onun üçün baza ilə integration test, yəni bir neçə hissənin birlikdə yoxlanması lazımdır.

Məşq. Konstruktorunda təsadüfi `Guid` yaradan `Probe` sinfi yaz. Guid 128 bitlik identifikatordur. Eyni scope-da iki dəfə, ikinci scope-da bir dəfə resolve et. Scoped üçün ilk iki ID eyni, sonuncu fərqli olmalıdır. Singleton-da hamısı eyni, transient-də hamısı ayrı olacaq. Sonra transient-i singleton sahəsində saxla: onun ID-si hər metod çağırışında dəyişməyəcək. Bu nəticə lifetime ilə real istinad ömrünün fərqini göstərir.

11 Rəsmi mənbələr

- Microsoft Learn - Dependency injection overview
- Microsoft Learn - Service lifetimes
- Microsoft Learn - Dependency injection guidelines
- Microsoft Learn - Architectural principles

Yoxlama tarixi 28 oktyabr 2026. Kodlar mövzunu göstərən fraqmentlərdir; `AppDbContext`, `Orders` və SQL implementasiyası layihədə ayrıca qurulur.

EF Core məlumat yükləmə növləri

EF Core-da əlaqəli məlumatı nə vaxt və necə yüklədiyin həm nəticənin düzgünlüyünü, həm sorğu sayını, həm də yaddaş sərfini dəyişir. Eager, explicit və lazy loading üç yükləmə üsuludur. Tracking isə ayrıca qərardır: yüklənmiş obyektin dəyişiklikləri izlənilsinmi? Bu iki qərarı qarışdırmamaq dərsin əsas məqsədidir.

İlkin materiallar `ef_core_tracking.pdf`, `files/04_ef_core.pdf` və `files/01_dotnet_core_fundamentals.pdf` idi. Explicit və lazy loading şərtləri, filtered Include, split query kompromisləri və yanlış performans ümumiləşdirmələri burada tamamlanır.

1 Modeli başa düşək

ORM, Object Relational Mapper, proqram obyektləri ilə əlaqəli verilənlər bazası cədvəlləri arasında uyğunluq quran vasitədir. EF Core .NET üçün ORM-dir. **Entity** bazada saxlanan məlumatı təmsil edən obyektdir. **Primary key**, əsas açar, sətri fərqləndirir. **Foreign key**, xarici açar, başqa cədvəlin açarına istinaddır. **Navigation property** obyekt modelində həmin əlaqə ilə başqa obyektə və ya obyektlər toplusuna keçid verir.

Bir `Order` sifarişinin bir `Customer` müştərisi və çoxlu `OrderItem` sətri olsun. `CustomerId` xarici açardır. `Customer` reference navigation, yəni bir obyektə keçiddir. `Items` collection navigation, yəni çoxlu obyektə keçiddir. İki cədvəlin şərtə görə birləşdirilməsi SQL-də **JOIN** adlanır.

C#

```
public class Order
{
    public int Id { get; set; }
    public int CustomerId { get; set; }
    public Customer Customer { get; set; } = null!;
    public List<OrderItem> Items { get; set; } = new();
}
```

Bu model fraqmentidir; `Customer`, `OrderItem` və DbContext ayrıca müəyyən edilir. `int` tam ədəd, `List<T>` dəyişə bilən siyahıdır. `get/set` xassənin oxunması və dəyişdirilməsidir. `null!` başlanğıcda boş istinada görə kompilyator xəbərdarlığını susdurur; runtime-da həqiqətən null olmayacağına zəmanət yaratmır. Boş `Items` siyahısı da əlaqənin bazadan yükləndiyini sübut etmir.

2 Sorğu nə vaxt işləyir

LINQ C#-da məlumat sorğularını ifadə edən metodlar və dil imkanlarıdır. `Where` filtr, `OrderBy` sıralama, `Select` seçilən nəticə formasını qurur. `IQueryable<T>` sorğunu provider-in tərcümə edə bildiyi ifadə kimi saxlayır. Provider burada C# ifadəsini SQL-ə çevirən EF Core hissəsidir. **Deferred execution** sorğunun qurulduğu anda deyil, nəticə istənəndə icrasıdır.

C#

```
var query = db.Orders.Where(o => o.CustomerId == 7);
// Hele netice materialize olunmayib.
var list = await query.ToListAsync(ct);
// SQL burada gonderilir ve obyektler yaranir.
```

Materialization bazadan gələn sətirlərin C# obyektlərinə çevrilməsidir. `ToListAsync` nəticəni siyahıya yığır. `await` asinxron əməliyyatın bitməsini gözləyir. `ct` ləğv siqnalını daşıyan `CancellationToken`-dir. `IEnumerable<T>` sadəcə elementləri gəzmək müqaviləsidir; “həmişə bütün məlumatı əvvəl RAM-a yükləyir” tərifi yanlışdır. `AsEnumerable()` özü sorğunu icra etmir, ancaq sonrakı LINQ to Objects əməliyyatlarını C#

tərəfində tətbiq edir. `ToList()` isə nəticəni yaddaşa yığır.

Sorğu obyektini DbContext bağlandıqdan sonra icra etmək olmaz. Context sorğunu göndərən və lazım gəlsə izləmə aparan iş vahididir. Onun ömrü nəticənin alınmasına çatmalıdır.

3 Eager loading əvvəlcədən əlaqələri seçir

Eager loading əsas nəticə alınanda lazım olan əlaqələri də sorğuya daxil etməkdir. `Include` bir səviyyəni, `ThenInclude` həmin əlaqənin davamını göstərir.

C#

```
var orders = await db.Orders
    .Include(o => o.Customer)
    .Include(o => o.Items)
        .ThenInclude(i => i.Product)
    .Where(o => o.CustomerId == 7)
    .ToListAsync(ct);
```

Bu nümunədə sifarişlər, onların müştərisi, sətirləri və həmin sətirlərin məhsulları tələb olunur. `Product` məhsul əlaqəsidir. Sorğu hansı əlaqələrin lazım olduğunu açıq göstərir. Ancaq eager loading həmişə bir SQL əmri demək deyil: single query rejimində adətən JOIN-lər, split query rejimində əlavə sorğular yarana bilər.

Bir müştərinin adı hər sifariş sətrində təkrar ötürülə bilər. Böyük mətn və ikili məlumat, məsələn şəkil, təkrarlanırsa şəbəkə yükü artır. “Əlaqə lazımdırsa bütün entity-ni Include et” əvəzinə nəticədə hansı sahələrin lazım olduğunu müəyyənləşdir.

4 Explicit loading sonradan açıq əmr verir

Explicit loading əsas entity alındıqdan sonra əlaqəni ayrıca əmrlə yükləməkdir. Seçim metodda aydın görünür. Məsələn, istifadəçi “sifariş detalları” düyməsini açdıqda sətirləri gətirə bilərsən.

C#

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42, ct);

await db.Entry(order)
    .Collection(o => o.Items)
    .LoadAsync(ct);

await db.Entry(order)
    .Reference(o => o.Customer)
    .LoadAsync(ct);
```

`SingleAsync` dəqiq bir nəticə gözləyir; sıfır və ya birdən çox nəticə xətadır. `Entry` entity barədə EF Core məlumatına çıxış verir. `Collection` çoxlu, `Reference` tək əlaqəni seçir. `LoadAsync` seçilmiş əlaqəni yükləyir. Tədris nümunəsi tracking aktiv olan context nəzərdə tutur; müxtəlif EF Core versiyalarında detached entity davranışını ayrıca yoxlamaq lazımdır.

Yalnız say lazımdırsa bütün sətirləri yükləmə:

C#

```
var count = await db.Entry(order)
    .Collection(o => o.Items)
    .Query()
    .CountAsync(ct);
```

`Query` əlaqəni IQueryable kimi verir, `CountAsync` bazada say hesablayır. Bu əməliyyat `Items` kolleksiyasının tam yüklənməsi demək deyil. Explicit loading nəzarəti artırır, amma dövrdə hər entity üçün çağırılsa yenə N+1 problemi yarada bilər.

5 Lazy loading xassəyə çatanda gizli yükləyir

Lazy loading əlaqəyə ilk müraciət zamanı onun lazım gəlsə avtomatik alınmasıdır. `order.Customer.Name` sətri görünüşcə yaddaş oxunuşudur, amma bazaya sorğu göndərə bilər. EF Core-da bu davranış adi modeldə avtomatik aktiv deyil.

Proxy üsulunda `Microsoft.EntityFrameworkCore.Proxies` paketi və `UseLazyLoadingProxies()` konfiqurasiyası istifadə edilir. **Proxy** davranışı tutmaq üçün yaradılmış əlavə obyekt tipidir. Sinifdən miras almaq mümkün olmalı, uyğun navigation xassələri `virtual` olmalıdır. `virtual` davranışın törəmə sinifdə əvəz edilməsinə imkan verir. Digər yanaşma `ILazyLoader` xidmətindən istifadədir; proxy tələbləri həmin üsul üçün eyni deyil.

C#

```
// Proxy usulunda modelden fraqment:
public virtual Customer Customer { get; set; } = null!;

// Konfiqurasiya fraqmenti:
options.UseLazyLoadingProxies().UseSqlServer(connection);
```

`connection` baza bağlantı parametridir. `UseSqlServer` SQL Server provider-ini seçir. Lazy loading üçün context-in işlək olması lazımdır. Cavab JSON-a çevrilərkən serializer navigation-ları oxuyub əlavə SQL sorğuları və ya dairəvi obyekt istinadları problemi yarada bilər. DTO qaytarmaq və lazım olan məlumatı açıq seçmək API-də daha nəzarətli yanaşmadır.

6 N+1 problemini sayla başa düşək

N+1 bir əsas sorğudan sonra N obyekt üçün ayrıca sorğu göndərilməsidir. 50 sifariş alıb hər birinin hələ yüklənməmiş sətirlərinə lazy keçsən, sadə ssenaridə 1 + 50 = 51 sorğu alınar. Hər şəbəkə gediş gəlişi 8 ms olsa, təkcə 50 əlavə gediş gəliş təxminən 400 ms gözləmə yaradar. Bu hesabi nümunədir, ölçülmüş benchmark deyil.

Eyni müştəri bir neçə sifarişdə təkrar istifadə olunursa tracking və əvvəl yüklənmiş əlaqələr faktiki sayın dəyişməsinə səbəb ola bilər. Buna görə “100 sifariş həmişə 101 sorğudur” hökmü yox, konkret yüklənmə vəziyyəti ilə izah edilməlidir. **Roundtrip** tətbiqdən bazaya gedib cavabın geri gəlməsi dövrüdür.

Həll olaraq eager loading, projection və ya toplu əlavə sorğu seçilə bilər. **Batch** bir çox obyekt üçün işi toplu görməkdir. `foreach` içində explicit `LoadAsync` yazmaq gizliliyi aradan qaldırır, amma sorğu sayını avtomatik azaltmır. SQL əmrlərinin logunu açıb real sayı ölçmək lazımdır.

7 Projection daha az məlumat seçir

Projection nəticəni lazım olan sahələrə çevirməkdir. DTO read-only, yəni yalnız oxunan ekran üçün tam entity qrafından daha uyğun ola bilər.

C#

```
var rows = await db.Orders
    .Where(o => o.CustomerId == 7)
    .Select(o => new
    {
        o.Id,
        CustomerName = o.Customer.Name,
        ItemCount = o.Items.Count
    })
    .ToListAsync(ct);
```

Burada `new { ... }` adsız nəticə tipi yaradır. EF Core uyğun ifadələri SQL-ə çevirə bilirsə müştəri adı və say bazada hesablanır; əvvəlcə bütün `Items` obyektlərini çəkmək lazım deyil. Bu projection üçün Include tələb olunmur. Lakin hər C# metodunun SQL qarşılığı yoxdur; tərcümə dəstəyi provider və versiyadan asılıdır. Entity obyekti daxil etməyən saf scalar projection-da tracker-ə entity əlavə olunmur. Scalar tək dəyərdir, məsələn ad və ya say.

8 Single query split query və çoxalma

Bir sifarişin 10 item-i və ayrıca 4 payment-i varsa eyni səviyyədə iki kolleksiyanın JOIN-i 10 x 4 = 40 nəticə sətri yarada bilər. 100 belə sifariş üçün 4000 sətir ötürülə bilər. **Cartesian explosion** bir-birindən müstəqil kolleksiyaların birləşməsi ilə nəticə sətirlərinin çoxalmasıdır. Nested, yəni biri digərinin içində olan əlaqəni, sibling, yəni eyni səviyyəli əlaqə ilə qarışdırma.

C#

```
var orders = await db.Orders
    .Include(o => o.Items)
    .Include(o => o.Payments)
    .AsSplitQuery()
    .ToListAsync(ct);
```

`AsSplitQuery` uyğun kolleksiya yükləməsini bir neçə SQL sorğusuna bölür. Dəqiq sorğu sayını “hər Include üçün bir” kimi əzbərləmək olmaz; reference əlaqələr, sorğu forması, provider və versiya təsir edir. Nəticənin çoxalması azalır, amma əlavə roundtrip və buffering, yəni aralıq nəticələri yaddaşda saxlama, xərci yarana bilər.

Birinci və ikinci SQL arasında başqa transaction məlumatı dəyişərsə bir-birinə tam uyğun olmayan anların məlumatı gələ bilər. Transaction bir neçə baza əməliyyatının müəyyən bütövlük qaydaları altında birləşdirilməsidir. Uyğun isolation, yəni paralel işlər arasında görünmə qaydası, lazım ola bilər və onun da xərci var. Səhifələmədə `OrderBy`-a unikal əlavə açar, məsələn `ThenBy(o => o.Id)`, vermək eyni tarixli sətirlərin qeyri-müəyyən sırasını aradan qaldırır.

9 Tracking yükləmə növü deyil

Change tracker yüklənmiş entity-lərin vəziyyətini və dəyişikliklərini izləyir. **Snapshot** ilkin dəyərlərin yadda saxlanan surətidir. **Identity resolution** eyni açarlı entity-nin eyni nəticə qrafında bir nüsxə kimi birləşdirilməsidir. **Navigation fixup** əvvəl yüklənmiş uyğun entity-lərin əlaqə xassələrinin avtomatik uyğunlaşdırılmasıdır.

`AsNoTracking` əsas context tracker-inə həmin sorğu entity-lərini əlavə etmir. Oxuma ssenarilərində faydalıdır, amma “həmişə 20-30 faiz sürətlidir” deyə sabit rəqəm yoxdur. Tracking əvvəl yüklənmiş entity-ni təkrar istifadə etməklə bəzi hallarda xərci azalda bilər. `AsNoTrackingWithIdentityResolution` isə əsas tracker-dən ayrı, nəticə daxilində təkrar nüsxələri birləşdirir.

Tracking ilə alınan entity dəyişdirilib `SaveChangesAsync` çağırılanda yenilənə bilər. No-tracking entity-ni dəyişmək təkbaşına UPDATE göndərmir. `Update(entity)` bütün dəyişdirilə bilən sahələri modified işarələyə bilər; disconnected, yəni izləmə mühitindən ayrılmış məlumat üçün yalnız icazəli sahələri attach və ya izlənən entity üzərinə köçürmək daha dəqiq ola bilər. `Attach` entity-ni tracker-ə tanıdır.

Filtered Include əlaqənin müəyyən hissəsini yükləyir: məsələn `Include(o => o.Items.Where(i => i.Price > 10))`. Əvvəl eyni tracking context-də başqa item-lar yüklənibsə fixup səbəbindən nəticə gözlədiyindən geniş görünə bilər. Üstəlik əlaqə loaded işarələnə bilər, halbuki onun yalnız bir hissəsi alınıb. Təmiz yeni context və ya məqsədə uyğun no-tracking sorğusu bu vəziyyəti idarə etməyə kömək edir.

10 Seçim və müsahibə məşqi

Ehtiyac	Başlanğıc seçim və səbəb
Siyahıda ad və say	Projection. Lazımsız entity və sütunları götürmür.
Əlaqələr əvvəlcədən bəllidir	Eager loading. Məlumat ehtiyacını bir yerdə açıq göstərir.
Bir obyektin detalları sonradan lazımdır	Explicit loading. Əlavə SQL çağırışı nəzarətdədir.
Navigation-a rahat keçid tələb olunur	Lazy loading mümkündür; gizli sorğu sayı və context ömrü ölçülməlidir.
Bir neçə böyük kolleksiya var	Single və split query-ni real məlumatla müqayisə et.

DbContext-i Task.WhenAll ilə paylaşa bilərəmmi? Eyni context-də eyni anda iki baza əməliyyatı işlətmə. Ardıcıl await et və ya həqiqətən müstəqil işlər üçün ayrıca context yarat. İki context yaratmaq onların avtomatik eyni transaction-da olması demək deyil.

Məşq. 20 sifariş üçün item sayını göstərən ekran qur. Əvvəl lazy navigation-a keç, sonra eager loading, sonra scalar projection yaz. Hər variantın SQL sayını, qayıdan sütunlarını və vaxtını qeyd et. Gözlənən nəticə: lazy variantda çoxsaylı əlavə sorğu riski, eager variantda tam item məlumatı, projection-da isə yalnız lazım olan say. Dəqiq vaxt rəqəmini ölçmədən elan etmə.

11 Rəsmi mənbələr

- Microsoft Learn - Loading related data
- Microsoft Learn - Eager loading
- Microsoft Learn - Explicit loading
- Microsoft Learn - Lazy loading
- Microsoft Learn - Single versus split queries
- Microsoft Learn - DbContext configuration

Yoxlama tarixi 28 oktyabr 2026. C# fraqmentləri EF Core relational provider-i nəzərdə tutur; yaradılan SQL-i konkret layihədə log və `ToQueryString()` ilə yoxlamaq lazımdır. `ToQueryString` sorğunun SQL təsvirini göstərir; bütün son icra əmrlərini görmək üçün log daha uyğundur.

Stored procedure və function fərqi

Bu dərs əsasən Microsoft SQL Server və onun T-SQL dilinə aiddir. SQL cədvəllərdəki məlumatı sorğulamaq və dəyişmək üçün dildir. T-SQL SQL Server-in dəyişənlər, prosedurlar və idarəetmə əmrləri əlavə edən SQL variantıdır. Eyni “function” sözünün PostgreSQL-də bəzi məhdudiyyətləri fərqlidir; bir sistemin qaydasını o birinə köçürmək olmaz.

İlkin materiallar `MSSQL_Functions_AZ.pdf`, `MSSQL_Functions_AdventureWorks.pdf`, `MSSQL_Tehsil_Materiallari.pdf` və onun ikinci variantıdır. İlk iki sənəddə funksiya və procedure müqayisəsi artıq var. Bu dərs həmin bazanı genişləndirir, qaydaların sərhədini, nümunələrdəki sütun uyğunsuzluğunu, paralel yazma riskini və plan iddialarını dəqiqləşdirir.

1 Rutin parametr və nəticə

Routine, rutin, bazada adla saxlanan və yenidən çağırılan kod vahididir. **Stored procedure** əməliyyat və ya əmrlər ardıcıllığı icra edən saxlanmış rutindir. **Function** SQL ifadəsinin bir hissəsi kimi istifadə edilə bilən dəyər və ya cədvəl qaytaran rutindir. **UDF**, User Defined Function, istifadəçinin yaratdığı funksiyadır; `SUM` kimi hazır funksiyadan fərqlənir.

Parametr çağırış zamanı verilən giriş və ya çıxış dəyişənidir. `@CustomerId` T-SQL parametr adıdır. **Scalar** bir dəyər, məsələn `42` və ya `15.50` deməkdir. **Result set** sorğudan qayıdan sətir və sütunlar toplusudur. **Schema** baza obyektlərini qruplaşdıran addır; `dbo.Orders` içində `dbo` schema, `Orders` cədvəldir. Schema-nı göstərmək obyektin dəqiq tanınmasına kömək edir.

Procedure “sifarişi təsdiqlə, ehtiyatı azalt, audit qeydi yaz” kimi əməliyyatlar üçün uyğundur. Function “bu məbləğin vergisi nədir?” və ya “bu müştərinin sifariş cədvəlini ifadə kimi ver” sualı üçün uyğundur. Audit baş verən əməliyyatın sonradan yoxlama üçün qeydə alınmasıdır.

2 Eyni meyarlarla müqayisə

Meyar	SQL Server davranışı
Çağırış	Procedure `EXEC dbo.Name ...`; scalar function `SELECT dbo.Name(...)`; table-valued function `FROM dbo.Name(...)`.
Nəticə	Procedure result set, OUTPUT parametr və tam ədəd return status verə bilər. Function müqaviləsindəki scalar və ya cədvəli qaytarır.
Sorğuya qoşulma	Function SELECT və ya FROM daxilində uyğun formada işlənir. Procedure normal halda JOIN mənbəyi kimi birbaşa yazılmır.
Məlumat dəyişmə	Procedure cədvəllərdə INSERT UPDATE DELETE edə bilər. T-SQL UDF baza vəziyyətini dəyişən yan təsirlər üçün istifadə edilmir.
Transaction və xəta	Procedure uyğun transaction və TRY/CATCH məntiqi daşıya bilər. T-SQL UDF daxilində bu imkanlar məhduddur.
Dinamik SQL və müvəqqəti cədvəl	Procedure istifadə edə bilər. T-SQL UDF daxilində dynamic SQL və temp table olmaz; table variable mümkündür.

Side effect, yan təsir, nəticə qaytarmaqdan əlavə xarici vəziyyəti dəyişməkdir. **Dynamic SQL** SQL mətninin işləmə vaxtında qurulub icra edilməsidir. **Temp table** `#Name` ilə müvəqqəti cədvəldir. **Table variable** `@t`

`TABLE (...)` formasında cədvəl tipli lokal dəyişəndir. Bu iki vasitənin optimizasiya və davranış fərqləri var, eyni şey sayılmır.

3 Scalar function nümunəsi

T-SQL

```
CREATE OR ALTER FUNCTION dbo.NetAmount
(
    @Gross decimal(12,2),
    @Discount decimal(12,2)
)
RETURNS decimal(12,2)
AS
BEGIN
    RETURN @Gross - @Discount;
END;
GO

SELECT dbo.NetAmount(100.00, 15.00) AS Net;
```

Nəticə `85.00` olur. `decimal(12,2)` ümumilikdə 12 rəqəm, vergüldən sonra 2 rəqəm saxlayan dəqiq ədədi tipdir. `RETURNS` nəticə tipini elan edir, `RETURN` funksiyanın dəyərini verir. `AS Net` nəticə sütununa ad verir. `GO` serverə göndərilən T-SQL əmri deyil; SSMS kimi alətlərdə ayrı batch sərhədidir. Batch birlikdə göndərilən əmrlər qrupudur. Tətbiq kodundan göndərəndə GO hissələrini ayrı əmrlərə bölmək lazımdır.

Bu sadə hesablamanın function-a çıxarılması həmişə vacib deyil. Hər çağırışda `Gross - Discount` da kifayət edə bilər. `NULL` naməlum və ya olmayan dəyərdirsə çıxma nəticəsi də NULL ola bilər. `COALESCE` ilk NULL olmayan dəyəri seçir, amma naməlum məbləği avtomatik sıfır saymaq biznes qaydasına uyğun olmalıdır.

Deterministic eyni giriş və eyni nəzərə alınan vəziyyətdə eyni nəticə verən davranışdır. **Nondeterministic** davranışda tarix vaxt və digər dəyişən amillər nəticəni dəyişə bilər. Bütün nondeterministic funksiyalar bütün kontekstlərdə eyni qayda ilə qadağan edilmir; konkret UDF məhdudiyyətini yoxlamaq lazımdır.

4 Cədvəl qaytaran iki function növü

TVF, Table Valued Function, cədvəl nəticəsi verir. **Inline TVF** bir SELECT ifadəsi ilə müəyyən edilir. Optimizer, yəni icra üsulunu seçən baza komponenti, onun məntiqini əsas sorğu ilə birlikdə təhlil edə bilir.

T-SQL

```
CREATE OR ALTER FUNCTION dbo.CustomerOrders
(
    @CustomerId int
)
RETURNS TABLE
AS
RETURN
(
    SELECT Id, CustomerId, Total
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId
);
GO

SELECT Id, Total
FROM dbo.CustomerOrders(7)
WHERE Total > 100;
```

Bu nümunə `dbo.Orders(Id, CustomerId, Total)` cədvəli mövcud olduğunu nəzərdə tutur. Funksiyanın nəticəsi xaricdə əlavə filtr və JOIN üçün istifadə olunur. **Multi-statement TVF** isə cədvəl dəyişəni elan edir, bir neçə əmrlə onu doldurur və qaytarır. Daha çox məntiqə imkan verir, lakin sətir sayının qiymətləndirilməsi və optimizasiya məhdudiyyətləri yarada bilər. Yeni versiyalardakı optimizasiya imkanları səbəbindən “həmişə yavaşdır” demək də doğru deyil.

5 Procedure üç fərqli nəticə verə bilər

T-SQL

```
CREATE OR ALTER PROCEDURE dbo.GetCustomerOrders
    @CustomerId int,
    @OrderCount int OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT Id, Total
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId;

    SELECT @OrderCount = COUNT(*)
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId;

    RETURN 0;
END;
GO

DECLARE @Count int, @Status int;
EXEC @Status = dbo.GetCustomerOrders
    @CustomerId = 7,
    @OrderCount = @Count OUTPUT;

SELECT @Count AS OrderCount, @Status AS Status;
```

Birinci SELECT sifarişlərdən result set verir. `@Count` OUTPUT parametr ilə sayı alır. `@Status` isə procedure-un return status-unu alır; bu nümunədə 0 uğuru bildirmək üçün seçilib. “Procedure heç nə qaytarmır” buna görə yanlış cümlədir. Eyni zamanda procedure `RETURN` ilə mətn və ya cədvəl qaytarmır, tam

ədəd status verir.

`DECLARE` dəyişən elan edir. `COUNT(*)` sətirləri sayır. `SET NOCOUNT ON` təsirlənən sətir sayı barədə ayrıca protokol mesajlarını azaldır; SELECT nəticəsini gizlətmir. Onu hər ssenaridə böyük sürət artımının səbəbi kimi göstərmək olmaz. İki SELECT arasında başqa transaction dəyişiklik edə bilər; nəticə ilə sayın eyni anı əks etdirməsi tələb olunursa uyğun transaction və isolation seçimi lazımdır.

6 Transaction və xəta sərhədi

Transaction bir neçə dəyişiklik üçün bütövlük sərhədidir. **Atomicity** onun əməliyyatlarının birlikdə təsdiqlənməsi və ya geri alınmasıdır. `COMMIT` təsdiqləyir, `ROLLBACK` geri alır. `TRY/CATCH` icra xətasını tutmaq üçün bloklardır. `THROW` xətanı çağırana ötürür. `XACT_STATE()` transaction-un aktiv və təsdiqlənə bilən olub olmadığını bildirir.

T-SQL

```
-- Bu fraqment oz transaction-unun sahibidir.
SET XACT_ABORT ON;
BEGIN TRY
    BEGIN TRANSACTION;

    UPDATE dbo.Orders
    SET Status = 'Confirmed'
    WHERE Id = @OrderId;

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH;
```

`XACT_ABORT ON` bir çox runtime xətasında transaction-un tam ləğv olunmasına səbəb olur; bütün xəta növləri üçün universal qayda deyil. Bu fraqmentdə `@OrderId` xaricdən verilmiş parametrdir və `Status` sütunu mövcuddur. UPDATE sıfır sətrə təsir edərsə bu avtomatik xəta deyil; lazım gəlsə `@@ROWCOUNT` yoxlanmalıdır. `@@ROWCOUNT` son əmrin təsir etdiyi sətir sayıdır.

Procedure başqa transaction içindən də çağırıla bilər. Daxili `COMMIT` xarici transaction-u müstəqil təsdiqləmir; iç-içə transaction sayğacı və sahiblik qaydaları nəzərə alınmalıdır. Bu sadə nümunəni ümumi nested transaction şablonu kimi köçürmək olmaz. Bir transaction xaricə göndərilmiş email-i geri ala bilmir.

7 Performans və təhlükəsizlik mifləri

Execution plan sorğunun hansı operatorlarla icra olunacağını göstərən plandır. **Plan cache** hazırlanmış planların təkrar istifadə üçün yaddaşda saxlandığı yerdir. Procedure çox vaxt ilk icrada uyğun plan hazırlayır, sonradan onu təkrar istifadə edə bilər. Schema və statistika dəyişiklikləri və başqa səbəblər recompilation, yəni yenidən plan qurulması yarada bilər. Parametrli adi SQL də plan təkrar istifadəsindən yararlana bilər. “Procedure əvvəlcədən tam kompilyasiya olunub həmişə daha sürətlidir” dəqiq deyil.

Parameter sniffing plan hazırlanarkən konkret parametr dəyəri əsasında qiymətləndirmə aparılmasıdır. Bir müştərinin 2, başqasının 2 milyon sifarişi varsa eyni plan hamı üçün optimal olmaya bilər. Bu normal optimizasiya mexanizminin bəzən uyğun olmayan nəticəsidir; həlli ölçmə və versiyanın imkanları ilə seçilir.

Scalar UDF böyük nəticədə hər sətir üçün əlavə iş yarada bilər. SQL Server-in uyğun versiya və şərtlərində scalar UDF inlining həmin məntiqi əsas plana daxil edə bilər. Inlining ayrıca function çağırışını ifadəyə

açmaqdır. Dəstək və uyğunluq olmadığı halda eyni qazancı gözləmək olmaz.

SQL injection istifadəçi mətninin məlumat əvəzinə icra edilən SQL hissəsinə çevrilməsidir. Procedure daxilində istifadəçi mətnini SQL-ə birləşdirmək yenə injection yarada bilər. Dəyərləri parametr kimi ötürmək lazımdır. `sp_executesql` dinamik mətnlə ayrıca parametr dəyərlərini qəbul edən sistem procedure-udur. Cədvəl və sütun adı kimi identifikatorlar adi dəyər parametri deyil; onların məhdud icazəli siyahısı və düzgün ad qaçırılması lazımdır.

İcazəni procedure səviyyəsində vermək müəyyən təhlükəsizlik dizaynlarında faydalıdır, amma “tam qorunma” zəmanəti deyil. İstifadəçinin həqiqətən hansı məlumatı oxuya bildiyi, dynamic SQL və icra konteksti ayrıca nəzərə alınmalıdır.

8 Seçim məşqləri və cavablar

Sifarişi və ehtiyatı birlikdə dəyişmək? Procedure və ya tətbiqin idarə etdiyi transaction məntiqli başlanğıcdır. Function yan təsirli iş vahidi üçün uyğun deyil.

Cədvələ JOIN edib təkrar istifadə olunan filtr vermək? Inline TVF uyğun ola bilər. Procedure birbaşa FROM mənbəyi kimi normal istifadə olunmur.

Hər sətirdə 18 faiz hesablamaq? Sadə SQL ifadəsi və ya uyğun scalar function mümkündür. Çox sətirdə planı ölç. Təkrar istifadə ehtiyacı ilə çağırış xərcini birlikdə qiymətləndir.

PostgreSQL-də də function məlumat dəyişə bilməzmi? Bu, SQL Server T-SQL UDF qaydasıdır. PostgreSQL funksiyaları müəyyən şərtlərdə yazma əməliyyatları apara bilər. Procedure və function transaction nəzarəti qaydaları da həmin sistemdə ayrıca öyrənilməlidir.

Məşq. 3 sifarişlik Orders cədvəlində TVF və procedure yaz. TVF-i WHERE ilə daralt. Procedure-un result set-ini, OUTPUT sayını və return status-unu ayrı oxu. Nəticə olaraq bu üç qaytarılma kanalını bir-birindən izah etməyi bacarmalısan.

9 İlkin SQL nümunələrində konkret düzəlişlər

`MSSQL_Functions_AZ.pdf` 7-ci səhifədə `AktivİşçiLər` inline TVF-i İşçiID, TamAd, Email, Maaş və ŞöbəAdı qaytarır. Ardınca gələn JOIN isə `AI.VəzifəID` istifadə edir. Bu sütun function nəticəsində olmadığı üçün həmin sorğu işləməz. İşçilər cədvəlində həqiqətən belə xarici açar varsa onu function-un SELECT siyahısına əlavə etmək lazımdır. Yoxdursa JOIN doğru əlaqə açarı ilə yenidən qurulmalıdır. TVF xarici sorğuya yalnız qaytardığı sütunları təqdim edir, daxildəki cədvəlin bütün sütunlarını avtomatik açmır.

Həmin sənədin 8-ci səhifəsində multi-statement hesabatda `OrtaMaaş` sütunu elan edilir, amma göstərilən INSERT və UPDATE-lər onu doldurmur. Bu halda sütun NULL qalır; hesabatda orta məbləğ görünəcəyi nəticəsi çıxmır. Mənalı orta hesabı üçün mənbə dəyəri və qruplaşdırma qaydası əlavə edilməli və ya işlənməyən sütun nəticədən çıxarılmalıdır. `AVG` orta qiymət hesablayan SQL funksiyasıdır; `GROUP BY` sətirləri seçilmiş açara görə qruplaşdırır.

Müştəri əlavə etmə nümunələrində əvvəl `IF EXISTS`, sonra INSERT ilə email təkrarı yoxlanır. İki paralel sorğu eyni anda “yoxdur” nəticəsini alıb ikisi də insert edə bilər. Qayda email-in unikal olmasıdırsa uyğun UNIQUE constraint və ya unique index bazada bunu qorumalıdır. Tətbiq duplicate xətasını istifadəçiyə uyğun cavaba çevirə bilər. Duplicate eyni açarın təkrarıdır. Adi əvvəlcədən yoxlama rahat mesaj verir, amma paralel yazmada constraint-i əvəz etmir. NULL və email müqayisə qaydaları da həmin biznes tələbinə görə müəyyən edilir.

10 Rəsmi mənbələr

- Microsoft Learn - CREATE FUNCTION
- Microsoft Learn - Create user defined functions
- Microsoft Learn - CREATE PROCEDURE

Yoxlama tarixi 28 oktyabr 2026. SQL nümunələri izah üçündür, real bazada icra edilməyib. Cədvəllər və parametrlər göstərilən şərtlərlə ayrıca hazırlanmalıdır.

SQL indeksləri və sütuna indeks verməyin təsiri

İndeks müəyyən axtarışları, birləşmələri və sıralamaları daha az məlumat oxumaqla yerinə yetirmək üçün saxlanan əlavə məlumat strukturudur. Onun faydası konkret sorğudan asılıdır. İndeks sütunun dəyərini dəyişmir və hər sorğunu avtomatik sürətləndirmir. Oxuma qazancının qarşılığında yazma, disk və yaddaş xərci yaranır.

Bu dərs `DatabaseİndexingHtml`, `GIST&GIN&BRIN_Index.html`, `files/05_sql.pdf`, `DB_Arxitektura.pdf` və SQL-in daxili işləməsi materiallarını dəqiqləşdirir. SQL Server və PostgreSQL bölmələri açıq ayrılır. Əvvəlki materiallardakı sabit sürət rəqəmləri real ölçmə kimi qəbul edilmir.

1 Əvvəlcə cədvəl sətir səhifə və açar

Cədvəl eyni sütun quruluşlu məlumatlar toplusudur. **Sətir** bir qeyddir. **Sütun** həmin qeydin bir xassəsidir. **Key**, açar, indeksdə axtarışı və sıralamanı müəyyən edən dəyər və ya dəyərlərdir. **Page**, səhifə, baza mühərrikinin məlumat saxlayıb oxuduğu blokdur. SQL Server-in adi rowstore məlumat səhifələri 8 KB ölçüdədir. Rowstore bir sətrin sütunlarını sətir əsaslı quruluşda saxlayır.

Bir səhifədə çoxlu sətir yerləşə bilər. Ona görə 1 milyon sətrə baxmaq mütləq 1 milyon disk oxunuşu demək deyil. **Buffer pool** baza səhifələrinin RAM-da saxlanan keşidir. RAM işləyən proqramların sürətli, dəyişkən yaddaşıdır. **Logical read** səhifəyə buffer üzərindən müraciətdir. **Physical read** məlumatı saxlama qurğusundan gətirmək ehtiyacıdır. Keşdə olan səhifə üçün yeni disk oxunuşu tələb olunmaya bilər.

İndeksin kitab kataloquna bənzətməsi axtarış ideyasını izah edir, amma texniki zəmanət vermir. Bir kitabın bir neçə nüsxəsini götürmək və kitabın çox hissəsini oxumaq fərqli xərclərdir; bazada da uyğun nəticə sayı və qaytarılan sütunlar vacibdir.

2 B plus tree necə axtarır

B+ tree balanslı, çoxşaxəli axtarış ağacıdır. **Root** başlanğıc qovşaq, **branch** aralıq qovşaq, **leaf** son səviyyədir. Aralıq səhifələr açar aralıqlarını göstərir. Axtarış hər səviyyədə uyğun aralığa keçir və yarpaq səviyyəsində dəyəri tapır. Balanslı olması yarpaqlara yolun təxminən eyni dərinlikdə qalmasını bildirir.

Məntiqi sxem

```
Root:          [1..5000] [5001..10000]
                  |            |
Branch:      [1..2500] ...  [7501..10000]
                  |
Leaf:       [101, 102, 103, 104, ...]
```

Bu sxem iki uşaqlı binary tree təsviri deyil. Real səhifədə çoxlu açar yerləşir, buna **fan-out**, şaxələnmə sayı deyilir. `O(log n)` axtarışın məlumat artdıqca yavaş böyüyən nəzəri xərcini ifadə edir. `O` böyümə sürətinin işarəsidir, millisanə ölçüsü deyil. Daha dəqiq baxışda ağacın şaxələnməsi və qaytarılan K sətirin xərci də var: yolu tapmaq və uyğun nəticələri oxumaq ayrı işdir.

“1 milyon sətir üçün log2 hesabladım, deməli dəqiq 20 disk oxunuşudur” düzgün deyil. Ağac binary deyil, səhifələr keşdə ola bilər, əlavə lookup lazım gələ bilər və nəticə çox sətrdən ibarət ola bilər. İndeksdə aralıq axtarışı uyğun başlanğıcı tapıb yarpaq səhifələri boyunca davam edə bilər.

3 SQL Server clustered və nonclustered

Clustered rowstore index-in yarpaq səviyyəsi cədvəlin məlumat sətirləridir. Buna görə cədvəldə bu strukturdan bir dənə ola bilər. Açar sırasına uyğun məntiqi quruluş verir; bütün səhifələrin diskdə fasiləsiz fiziki qonşu qalmasına zəmanət vermir. SQL nəticəsinin sırası üçün yenə `ORDER BY` yazılır.

Heap SQL Server-də clustered index-i olmayan cədvəldir. Heap “bütün SQL sistemlərində bütün cədvəllər həmişə belə saxlanır” demək deyil. **Nonclustered index** ayrıca quruluşdur: açarları və sətri tapmaq üçün locator saxlayır. Locator heap-də row identifier, clustered cədvəldə isə clustered açar əsasında işləyir. Locator sətrə çatmaq üçün göstəricidir.

Key lookup nonclustered indeksdən tapılan clustered açarla əsas məlumat sətrinə əlavə gedişdir. **RID lookup** heap sətrinə əlavə gedişdir. Bir nəticə üçün ucuz olan lookup minlərlə nəticə üçün baha ola bilər. Clustered açar genişdirsə nonclustered indekslərin saxladığı locator xərci də arta bilər.

Primary key sətrlərin unikal və NULL olmayan identifikatorunu tələb edən məntiqi constraint-dir. Constraint məlumat üçün məcburi qaydadır. SQL Server çox vaxt ona dəstək üçün indeks yaradır, amma primary key mütləq clustered olmalıdır demək deyil. Cədvəlin ayrıca clustered açarı, primary key-in isə nonclustered indeksi ola bilər.

4 Sütuna indeks verəndə nə dəyişir

T-SQL

```
CREATE NONCLUSTERED INDEX IX_Orders_CustomerId
ON dbo.Orders(CustomerId);

SELECT Id, Total
FROM dbo.Orders
WHERE CustomerId = 7;
```

İndeks yaradılarkən mövcud uyğun məlumat oxunur, indeks qurulur və gələcək dəyişikliklərlə saxlanır. Sorğu gələndə **optimizer** mümkün planların təxmini xərcini qiymətləndirir. **Statistics**, statistika, dəyərlərin paylanması haqqında məlumatdır və neçə sətir gözlənildiyini təxmin etməyə kömək edir. İndeksin olması onun mütləq seçilməsi demək deyil.

Index seek uyğun açar və ya aralıqdan axtarışa başlayır. **Index scan** indeksin böyük hissəsini və ya hamısını gəzir. **Table scan** cədvəl məlumatını gəzir. Seek həmişə yaxşı, scan həmişə pis deyil: nəticə cədvəlin çox hissəsidirsə scan daha az iş görə bilər. Seek də geniş aralıq və çox lookup ilə bahalı ola bilər.

SELECT çox sürətlənə bilər. Amma `INSERT` yeni sətrin indeks yazısını da əlavə edir. İndekslənmiş açarın dəyişdiyi `UPDATE` indeksin uyğun hissələrini də dəyişir. `DELETE` indeks yazılarının təmizlənməsi işini yaradır. İndeks disk sahəsi tutur, aktiv səhifələri RAM-da yer tutur və dəyişikliklər transaction log yükünü artırır. Transaction log bərpa üçün dəyişiklik qeydlərini saxlayır. Yazma xərci bütün UPDATE-lərdə eyni deyil; indeksin əhatə etdiyi sütunlar və strukturdan asılıdır.

5 Composite indeksdə sıra niyə vacibdir

Composite index bir neçə sütundan ibarət açarı olan indeksdir. `(CustomerId, CreatedAt)` əvvəl müştəriyə, onun daxilində tarixə görə sıralanan telefon kitabçasına bənzəyir. `CustomerId = 7` şərti və tarix aralığı birlikdə bu quruluşa uyğundur. Yalnız tarixlə axtarışda isə ilk sütun məlum olmadığı üçün eyni səmərəli dar axtarış mümkün olmaya bilər.

T-SQL

```
CREATE INDEX IX_Orders_Customer_Date
ON dbo.Orders(CustomerId, CreatedAt DESC)
INCLUDE(Total, Status);

SELECT CreatedAt, Total, Status
FROM dbo.Orders
WHERE CustomerId = @CustomerId
  AND CreatedAt >= @From
ORDER BY CreatedAt DESC;
```

`@From` başlanğıc tarix parametridir. `DESC` azalan sıra deməkdir. **INCLUDE** sütunları indeksin yarpaq səviyyəsində nəticə üçün saxlayır; açarın axtarış sırasını genişləndirmir. **Covering index** konkret sorğunun ehtiyac duyduğu bütün məlumatı daşıyan indeksdir. “Covering” indeksin bütün sorğulara aid daimi xüsusiyyəti deyil.

Bu nümunədə CustomerId bərabərlik şərti, CreatedAt aralıq və sıralama şərtidir. “Ən çox unikal dəyəri olan sütun həmişə birinci olmalıdır” universal qayda deyil. Hədəf sorğuların bərabərlik və aralıq şərtləri, ORDER BY, paylanma və başqa sorğular birlikdə nəzərə alınır. `(Email, Status)` və `(Status, Email)` eyni bərabərliklə istifadə olunan bir sorğuda yaxın nəticə verə, başqa sorğularda fərqlənə bilər.

6 Unique filtered və aşağı cardinality

Unique index eyni açar dəyərinin təkrarına icazə vermir. Bu yalnız sürət üçün deyil, məlumat qaydasını qorumaq üçündür. **Filtered index** SQL Server-də yalnız şərtə uyğun sətirləri saxlayır. PostgreSQL-də oxşar anlayış partial index adlanır. Bu sintaksislər bütün xüsusiyyətlər üzrə tam eyni deyil.

T-SQL

```
CREATE INDEX IX_Orders_Pending
ON dbo.Orders(CreatedAt)
INCLUDE(CustomerId, Total)
WHERE Status = 'Pending';
```

Cardinality kontekstdən asılı olaraq fərqli dəyərlərin sayını və ya plan operatorunun nəticə sətir sayını bildirə bilər. **Predicate selectivity** filtrin məlumatın nə qədərini seçdiyini bildirir. 1 milyon sətrin 100-ü Pending-dirsə pay 0.01 faizdir və bu dar hissə üçün indeks faydalı ola bilər. İki mümkün status olması həmin nadir dəyəri indeksləşdirməyi avtomatik mənasız etmir.

Bool, iki qiymətli sütun, üçün indeks “həmişə faydasızdır” hökmü doğru deyil. Nadir dəyər, filtered indeks və uyğun covering sorğu fayda verə bilər. Eyni zamanda kiçik cədvəl üçün belə unique qaydanı qoruyan indeks məntiqli ola bilər. Sətir sayı üçün “1000-dən aşağı heç vaxt indeks qoyma” sərt hədd yoxdur.

7 SARGability və funksiyanın təsiri

Predicate sətirin uyğun olub olmadığını yoxlayan şərtdir. **SARGable** şərt indeksdə səmərəli axtarış aralığı yaratmağa uyğun yazılıb deməkdir. Adı Search ARGument ifadəsindən gəlir. Sütunu funksiya içində hesablamaq sadə indeksin dar seek kimi istifadəsinə mane ola bilər.

T-SQL

```
-- Tarix sutununa funksiya tetbiq edilir:
WHERE YEAR(CreatedAt) = 2026

-- Eyni ili yarimaciq interval ile secirik:
WHERE CreatedAt >= '20260101'
  AND CreatedAt <  '20270101'
```

Yarımaçıq interval başlanğıcı daxil edir, son sərhədi daxil etmir. Beləcə günün son saniyəsi və tarix dəqiqliyi problemi yaranmır. `YEAR` tarixdən ili çıxaran funksiyadır. Funksiya həmişə bütün indeks istifadəsini tam qadağan etmir; uyğun computed column indeksi və optimizer çevirmələri ola bilər. **Computed column** başqa sütunlardan hesablanan sütundur və indeks üçün ayrıca qaydalara tabedir.

`LIKE 'Ali%'` prefiks, başlanğıc hissə axtarışıdır və uyğun şəraitdə B-tree aralığına çevrilə bilər. `LIKE '%Ali%'` adi B-tree ilə eyni dar aralığı yaratmır. **Collation** mətnin sıralanma və hərf müqayisəsi qaydalarıdır. Böyük kiçik hərf, dil və operator qaydaları indeks seçiminə təsir edə bilər. İki fərqli tip müqayisə olunanda **implicit conversion**, gizli tip çevrilməsi, sütun tərəfində əlavə hesablama yaradaraq axtarışı çətinləşdirə bilər.

8 PostgreSQL indeksləri ayrıca dünyadır

PostgreSQL cədvəli heap quruluşunda saxlayır və indekslər sətir versiyalarına istinad edir. SQL Server clustered index məntiqini buraya birbaşa köçürmə. PostgreSQL-də `CLUSTER` cədvəli indeks sırasına uyğun yenidən yaza bilər, amma sonrakı yazmalar həmin sıranı avtomatik daimi saxlamır.

Tip	Məntiq və uyğun nümunə
B-tree	Sıralana bilən açarlarda bərabərlik və aralıq axtarışı. Adi standart seçimdir.
Hash	Hash kodla sadə bərabərlik axtarışı. Adi aralıq və ORDER BY üçün nəzərdə tutulmayıb.
GIN	Çox hissəli dəyərdən hissəyə geri xəritə saxlayır. JSONB, massiv və mətn tokenləri üçün uyğundur.
GiST	Müxtəlif tip və operatorlar üçün genişləndirilən axtarış çərçivəsidir. Həndəsi yaxınlıq və aralıqlar nümunədir.
SP-GiST	Məkanı hissələrə ayıran quruluşları dəstəkləyən çərçivədir. Məlumat tipinə uyğun operator sinfi tələb edir.
BRIN	Səhifə aralıqları haqqında xülasə saxlayır. Fiziki yerləşmə ilə dəyər arasında əlaqə güclüdürsə çox kiçik indekslə böyük hissələri atlaya bilər.

Hash dəyərdən hesablanan yığcam koddur; müxtəlif dəyərlər eyni koda düşə bilər və dəqiqlik yoxlaması lazım ola bilər. **GIN**, Generalized Inverted Index, məsələn “sql” etiketindən həmin etiketi daşıyan sətirlərə keçid saxlayır. **JSONB** PostgreSQL-in emala uyğun ikili JSON saxlanma tipidir. **GiST**, Generalized Search Tree, bütün operatorlara eyni sürət zəmanəti vermir. Operator class konkret tip üçün indeksin dəstəklədiyi müqayisələri müəyyən edir.

BRIN, Block Range Index, təkcə hər sətrin və ya hər səhifənin deyil, səhifə aralığının xülasəsini saxlayır. Min/max geniş yayılmış xülasə üsuludur. Namizəd aralıqlar tapılanda həqiqi sətirlər yenidən yoxlanır; buna **recheck** deyilir. Dəyərlərin fiziki sıra ilə korrelyasiyası, yəni birlikdə dəyişmə əlaqəsi, faydanı müəyyən edir. Tam ideal sıralama məcburi deyil.

PostgreSQL

```
CREATE INDEX ix_users_email_lower
ON users (lower(email));

CREATE INDEX ix_events_created_brin
ON events USING brin(created_at);

CREATE INDEX ix_posts_tags
ON posts USING gin(tags);
```

Bu alternativ nümunələr uyğun `users`, `events`, `posts` cədvəlləri və tipləri tələb edir. Birincisi expression index-dir: funksiyanın hesabladığı dəyəri indeksləşdirir. `lower` mətni kiçik hərfə çevirir. GIN üçün `tags` uyğun massiv tipi ola bilər. SQL Server-də hash indeksini adi disk rowstore B-tree-yə universal alternativ kimi yazmaq olmaz; onun memory-optimized, yaddaş üçün optimallaşdırılmış cədvəl xüsusiyyətləri ayrıca mövzudur.

9 Planı necə qiymətləndirməli

SQL Server-də Actual Execution Plan real icra barədə məlumat, `SET STATISTICS IO ON` oxunuş, `SET STATISTICS TIME ON` CPU və vaxt göstəriciləri verir. CPU prosessorun hesablamaya sərf etdiyi resursdur. **Elapsed time** istifadəçinin gözlədiyi keçən vaxtdır və bloklanma və şəbəkədən də təsirlənə bilər. Estimated plan icra etmədən təxmin göstərir.

PostgreSQL-də `EXPLAIN` planı, `EXPLAIN (ANALYZE, BUFFERS)` isə həqiqətən icra edib vaxt və buffer göstəricilərini göstərir. ANALYZE ilə yazma sorğusu da həqiqətən işləyir; onu təhlükəsiz test məlumatında yoxlamaq lazımdır. Index-only scan üçün indeksdə lazımi sütunların olması ilə yanaşı görünürlük məlumatı da təsir edir; heap-ə heç vaxt getmir demək olmaz.

Planın cost faizi real saniyə ölçüsü deyil. **Estimated rows** təxmin edilən, **actual rows** həqiqi sətir sayıdır. Böyük fərq paylanma, statistika və parametr uyğunluğu problemini göstərə bilər. Missing index tövsiyəsi də avtomatik əmr deyil; oxşar indekslər, yazma xərci və bütün iş yükü nəzərə alınmalıdır.

İndeks yoxlamasının ardıcıllığı: hədəf sorğunu və parametrləri seç, ilkin plan və oxunuşu qeyd et, bir namizəd indeks yarat, eyni şəraitdə yenidən ölç, yazma xərcini və başqa sorğuları nəzərdən keçir. Warm cache və cold cache, yəni məlumatın yaddaşda olması və olmaması, fərqli nəticə verə bilər. Laboratoriya nəticəsini bütün sistemlər üçün zəmanət kimi təqdim etmə.

10 Müsahibə məşqi

Email sütununa indeks qoyanda nə baş verir? Email-ə görə ayrıca axtarış quruluşu yaradılır. Uyğun sorğuda az səhifə oxumaq mümkün ola bilər, amma əlavə saxlanma və yazma işi yaranır. Unique seçilibsə təkrar email-lər də məhdudlaşdırılır.

Niyə indeks var amma istifadə edilmir? Nəticə çox genişdir, cədvəl kiçikdir, filtr uyğun aralıq yaratmır, statistika yanılır, tip çevrilir və ya daha ucuz başqa plan vardır. Mövcud planı görmədən “baza səhv edir” demək olmaz.

Məşq. 100 min sifarişdə `CustomerId` və tarix üzrə son 20 sifarişi seç. Əvvəl indeks olmadan, sonra tək CustomerId, sonra `(CustomerId, CreatedAt)` və uyğun INCLUDE ilə planı müqayisə et. Sıra üçün eyni tarixə qarşı `Id` əlavə açarı da düşün. Nəticəni “seek çıxdı” ilə deyil, oxunan səhifə, qayıdan sətir və yazma xərci ilə qiymətləndir.

11 Rəsmi mənbələr

- Microsoft Learn - SQL Server index types
- Microsoft Learn - Index maintenance and page density
- PostgreSQL - Index types

Yoxlama tarixi 28 oktyabr 2026. Ölçü və sürət rəqəmləri yalnız izah üçün veriləndə bunu mətn açıq bildirir. SQL nümunələri real bazada icra edilməyib.

SQL fraqmentasiya və defraqmentasiya

Fraqmentasiya məlumat və indeks səhifələrinin iş üçün əlverişsiz yerləşməsi və ya içəridə boş sahənin artması ilə bağlı problemləri təsvir edir. Defraqmentasiya həmin quruluşu daha uyğun formaya gətirmə işidir. Sözlər oxşar olsa da SQL Server indeks fraqmentasiyası, PostgreSQL bloat və əməliyyat sisteminin disk defraqmentasiyası eyni əməliyyat deyil.

İlkin qovluqda bu mövzu üçün müstəqil və əhatəli SQL fraqmentasiya PDF-i tapılmadı. GC materialındakı yaddaş fraqmentasiyası SQL indeks baxımı ilə eyni mövzu deyil. Buna görə bu dərs ayrıca əlavə edildi.

1 Səhifə və məntiqi sıra

SQL Server rowstore indeksində məlumat 8 KB-lıq **page**, səhifə bloklarında saxlanır. **B+ tree** axtarış üçün balanslı çoxşaxəli ağacdır. Onun **leaf**, yarpaq, səviyyəsi açar sırasına görə gəzməyə imkan verir. **Logical order** açarların tələb etdiyi sıra, **physical order** səhifələrin fayl daxilindəki yerləşmə sırasıdır.

Məsələn, açar ardıcıllığında səhifələr A, B, C olmalıdır. Faylda yerləşmə A, başqa səhifələr, C, başqa səhifələr, B kimidirsə məntiqi gediş fiziki olaraq ardıcıl olmaya bilər. Burada söhbət cədvəldə yanlış nəticə saxlanmasından getmir. Fraqmentasiya məlumatın korlanması, yəni corruption deyil.

2 Xarici daxili və fayl səviyyəli problem

Logical fragmentation, çox vaxt xarici fraqmentasiya deyilən vəziyyət, indeksin məntiqi sıra ilə fiziki səhifə ardıcıllığının uyğun gəlməməsidir. Geniş aralıq scan-lərində ardıcıl oxuma və read-ahead səmərəliliyinə təsir edə bilər. **Scan** çoxlu səhifəni gəzməkdir. **Read-ahead** baza mühərrikinin lazım olacağını gözlədiyi səhifələri əvvəlcədən gətirməsidir.

Page density səhifənin doluluq nisbətidir. Səhifə daxilində çox boşluq olması daxili fraqmentasiya kimi izah edilir. Eyni məlumat daha çox səhifəyə yayılırsa daha çox oxunuş, RAM və keş sahəsi lazım ola bilər. Boşluq 8 KB səhifənin hər baytının istifadəçi məlumatı olması ilə hesablanmır; səhifənin idarəetmə xərcləri də vardır.

Hesabi nümunə: bir məlumat toplusu yüksək doluluqda 1000 səhifəyə, yarı dolu vəziyyətdə təxminən 2000 səhifəyə yayıla bilər. Dəqiq say sətir ölçüsü və struktur xərclərindən asılıdır. Məqsəd “fraqmentasiya 50 faizdirsə sorğu iki dəfə yavaşdır” demək deyil; eyni məlumat üçün daha çox səhifə daşıma ehtiyacını göstərməkdir.

File system fragmentation isə baza faylının əməliyyat sisteminin saxlama bloklarında yayılmasıdır. SQL `ALTER INDEX` əmri ilə diskin ümumi defraqmentasiyası eyni şey deyil. SSD, yarımkeçirici saxlama qurğusu, ilə HDD, mexaniki disk, təsirləri də eyni deyil. SSD-də fiziki ardıcıllıq problemi zəifləyə bilər, amma aşağı page density-nin oxunuş və yaddaş xərci qalır.

3 Page split və yazmalar

Page split səhifədə yer çatmayanda məlumatın başqa səhifəyə bölüşdürülməsidir. Aralıq açarlarla dolu bir səhifəyə yeni sətir əlavə olunanda yeni səhifə lazım ola bilər. Yeni səhifənin faylın başqa yerində yerləşməsi məntiqi ardıcıllığı poza, iki səhifədə boşluq yarada və əlavə log işi doğura bilər.

`DELETE` boş sahə yarada bilər. Dəyişən uzunluqlu sütunun böyüdüyü `UPDATE` yer çatışmazlığı yarada bilər. Təsadüfi clustered açarlar ağacın müxtəlif yerlərinə insert paylayır və müəyyən iş yüklərində split-ləri artıra bilər. Artan açarlar isə son səhifədə **contention**, yəni eyni resurs uğrunda sıx paralel rəqabət yarada bilər.

Ona görə açar seçimi təkcə “təsadüfi pis, artan yaxşı” hökmü ilə həll olunmur.

Fill factor indeks yenidən qurularkən yarpaq səhifələrini hansı hədəf doluluqla yaratmağı bildirir. Məsələn, 90 gələcək aralıq insert-lər üçün təxminən boş yer saxlayır. Bu, hər səhifənin gələcəkdə daim 90 faiz qalması zəmanəti deyil. Aşağı fill factor bəzi split-ləri azaldır, amma başlanğıcdan daha çox səhifə və oxunuş yaradır. Ölçmədən bütün indekslərə aşağı dəyər tətbiq etmək olmaz.

4 SQL Server-də nəyi ölçürük

DMV, Dynamic Management View, serverin iş vəziyyəti barədə məlumat verən sistem görünüşüdür. `sys.dm_db_index_physical_stats` fiziki indeks ölçülərini təqdim edən sistem funksiyasıdır. `avg_fragmentation_in_percent` sıra uyğunsuzluğu, `avg_page_space_used_in_percent` doluluq, `page_count` səhifə sayı üçün istifadə edilir.

T-SQL

```
SELECT
    i.name,
    s.partition_number,
    s.page_count,
    s.avg_fragmentation_in_percent,
    s.avg_page_space_used_in_percent
FROM sys.dm_db_index_physical_stats
(
    DB_ID(), OBJECT_ID(N'dbo.Orders'),
    NULL, NULL, 'SAMPLED'
) AS s
JOIN sys.indexes AS i
  ON i.object_id = s.object_id
 AND i.index_id = s.index_id
WHERE s.index_level = 0
  AND s.alloc_unit_type_desc = 'IN_ROW_DATA'
  AND i.type IN (1, 2);
```

Bu diaqnostika yalnız `dbo.Orders` üçün nümunədir. `DB_ID()` cari baza ID-sini, `OBJECT_ID` obyekt ID-sini verir. **Partition** cədvəl və ya indeksin bölünmüş hissəsidir; hissələr ayrıca fərqli vəziyyətdə ola bilər. `index_level = 0` yarpaq səviyyəsidir. `IN_ROW_DATA` sətir daxilində saxlanan məlumatın ayırma vahidid ir. `type IN (1,2)` rowstore clustered və nonclustered indeksləri seçir.

`SAMPLED` nümunə götürərək analiz edir; kiçik indeksdə daha tam analizə keçə bilər. `LIMITED` daha yüngül məlumat verir və bəzi doluluq göstəriciləri NULL ola bilər. `DETAILED` daha çox səhifə oxuyur. Bu sorğular da resurs və kilid yükü yarada bilər. Geniş server analizi əvəzinə hədəf indeksdən başlamaq daha ölçülə bilən yanaşmadır. Göstəricilərə çıxış üçün uyğun server və baza icazəsi tələb olunur.

5 Reorganize nə edir

REORGANIZE rowstore indeksinin yarpaq səviyyəsində səhifə sırasını və sıxlığını yaxşılaşdıran daha mərhələli baxımdır. Mövcud indeks üzərində işləyir. Tam yenidənqurmadan az resurs tələb edə bilər, lakin çox böyük problem üçün daha uzun çəkə bilər.

T-SQL

```
ALTER INDEX IX_Orders_Customer_Date
ON dbo.Orders REORGANIZE;
```

Bu əmr indeksi yeni adla yaratmır. Adətən online xarakterlidir, yəni xidmət davam edə bilər, amma resurs və kilid təsirini sıfırlamır. **Lock** verilənlərə paralel giriş qaydasını qoruyan kiliddir. REORGANIZE indeks

statistikasını ayrıca yeniləmir. Statistics dəyərlərin paylanmasını təsvir edir və optimizer-in plan qərarına təsir edir. Lazımdırsa statistikaya ayrıca baxım qərarı verilir.

6 Rebuild nə edir

REBUILD indeksi yenidən qurur. Uyğun səhifə sırası və seçilmiş fill factor ilə yeni quruluş yaranır. CPU, I/O, əlavə disk, transaction log və bəzi ssenarilərdə `tempdb` xərci yüksək ola bilər. I/O disk və şəbəkə giriş çıxışıdır. `tempdb` SQL Server-in aralıq işlər üçün istifadə etdiyi sistem bazasıdır.

T-SQL

```
ALTER INDEX IX_Orders_Customer_Date
ON dbo.Orders REBUILD;
```

Offline rebuild istifadəçi girişini müəyyən müddət bloklaya bilər. **Online rebuild** uyğun xüsusiyyət olduqda daha çox paralel girişə imkan verir, amma başlanğıc və son mərhələlərdə kilidlər yenə mümkündür. Versiya, edition, indeks tipi və digər məhdudiyyətlər yoxlanmadan `ONLINE = ON` hər yerdə işləyəcək sayılmır. **Edition** məhsulun imkan paketidir.

Uyğun versiya və indekslərdə **resumable rebuild** işi dayandırıb davam etdirməyə imkan verir. Bunun şərtləri ayrıca yoxlanmalıdır. Bütün indeksi rebuild etmək həmin indeks statistikasını da yeniləyir; amma cədvəlin bütün digər sütun statistikalarının hamısını avtomatik yeniləmir. Partition rebuild və başqa xüsusiyyətlərdə statistika təfərrüatı ayrıca nəzərə alınır.

7 Hansı halda hansı əməliyyat

Vəziyyət	Düşünüləcək qərar
Kiçik indeks yüksək faiz göstərir	Səhifə sayı azdırsa praktiki qazanc olmaya bilər. Faizə baxıb avtomatik baxım etmə.
Böyük aralıq scan yavaşdır	Fraqmentasiya, density, storage və oxunuşu birlikdə qiymətləndir.
Tək açarla dar seek normaldır	Yüksək fraqmentasiya təkbaşına təcili rebuild səbəbi deyil.
Səhifələr çox boşdur	Density yaxşılaşmasının oxunuş və yaddaş qazancını qiymətləndir.
Plan yanlış sətir sayı hesablayır	Statistikaya və parametrə bax; indeks quruluşu əsas səbəb olmaya bilər.
Yazma yükü çoxdur	Baxımın log, replika gecikməsi, kilid və əlavə split təsirini hesabla.

Tarixi 5 faiz və 30 faiz kimi hədlər bəzi köhnə şablonlarda görünür. Bunlar bütün sistemlər üçün məcburi qərar qaydası deyil. Microsoft-un aktual yanaşması iş yükü, page density və ölçülən performans qazancına əsaslanmağı vurğulayır. **Workload** sistemin real oxuma və yazma əməliyyatları toplusudur.

İş axını belədir: yavaş sorğunu ölç, indeks ölçüsünü və doluluğu yoxla, səbəbi təsdiqlə, bir indeksdə seçilmiş baxımı apar, sonra eyni sorğunu müqayisə et. **Maintenance window** istifadəçilərə təsirin qəbul olunan aşağı yük vaxtıdır. Disk və log sahəsi, bloklanma, replikasiya gecikməsi və işi dayandırma imkanları nəzərə alınır. Replika başqa serverdə saxlanan məlumat nüsxəsidir.

8 PostgreSQL bloat VACUUM və REINDEX

PostgreSQL **MVCC**, Multi Version Concurrency Control, ilə eyni sətrin müxtəlif versiyalarını saxlayaraq paralel oxuma və yazmanı idarə edir. Artıq heç bir aktiv transaction-a lazım olmayan köhnə sətir versiyası **dead tuple** adlanır. Tuple burada sətir versiyasıdır. **Bloat** cədvəl və ya indeksin faydalı məlumatına nisbətən lazımsız böyüməsidir.

VACUUM təmizlənə bilən köhnə versiyaları idarə edir və sahəni gələcək təkrar istifadə üçün azad edir. Adi VACUUM ümumən faylı sıxıb bütün boşluğu əməliyyat sisteminə qaytarmır; müəyyən son boş səhifələr üçün istisnalar ola bilər. **ANALYZE** planlayıcı statistikalarını yeniləyir. **Autovacuum** bu işləri avtomatik görən fon mexanizmidir.

VACUUM FULL cədvəli yenidən yazaraq sıxlaşdıra və yer qaytara bilər, amma güclü kilid və əlavə sahə tələb edir. **REINDEX** indeksin yeni nüsxəsini qurur. `REINDEX ... CONCURRENTLY` uyğun hallarda yazmaları daha az bloklamağa çalışır, əvəzində əlavə vaxt və iş görür. Məhdudiyyət və versiya qaydaları var.

PostgreSQL

```
VACUUM (ANALYZE) public.orders;

REINDEX INDEX CONCURRENTLY public.ix_orders_customer_date;
```

Bu iki əmr eyni işi görmür. Birincisi cədvəlin versiya təmizliyi və statistikaya, ikincisi indeksin yenidən qurulmasına aiddir. SQL Server `REORGANIZE` əmrinin PostgreSQL-də eyni adlı qarşılığı kimi təqdim edilməməlidir. Uzun transaction-lar köhnə versiyaların təmizlənməsini gecikdirə bilər; təkcə REINDEX kök səbəbi həll etməyə bilər.

9 Columnstore ayrıca qiymətləndirilir

Columnstore məlumatı sütun əsaslı və sıxılmış qruplarda saxlayan indeks ailəsidir. Analitik hesabatlarda çox sətirdən az sütun oxumaq üçün faydalıdır. **Rowgroup** birlikdə saxlanan sətir qrupudur. **Delta store** hələ əsas sıxılmış qrupa keçməmiş məlumatın saxlandığı hissədir. Silinmiş sətir faizi və kiçik rowgroup-lar burada baxım qərarına təsir edir.

Rowstore B+ tree-dəki `avg_fragmentation_in_percent` məntiqini columnstore-a olduğu kimi tətbiq etmək olmaz. Columnstore REORGANIZE və REBUILD semantikası da öz xüsusiyyətlərinə malikdir. Buna görə əvvəl indeksin tipini müəyyən et, sonra uyğun ölçünü seç.

10 Müsahibə cavabı və məşq

Fraqmentasiya niyə yavaşıda bilər? Məntiqi sıra ilə fiziki səhifə sırası pozulduqda geniş oxumalar daha səmərəsiz ola bilər. Səhifə doluluğu aşağı olduqda eyni məlumat üçün daha çox səhifə oxunur və RAM tutulur. Təsirin ölçüsü sorğudan və storage-dən asılıdır.

Rebuild hər yavaş bazanı düzəldirmi? Xeyr. Yanlış indeks, bloklanma, bahalı JOIN, yanlış statistika və ya tətbiqdə N+1 səbəb ola bilər. Baxım yalnız uyğun fiziki quruluş problemini hədəfləyir.

Məşq. Test bazasında bir indeks üçün page_count, density və fragmentation göstəricilərini qeyd et. Aralıq SELECT-in logical read və vaxtını ölç. Reorganize və ya rebuild-dən sonra ölçməni təkrarla. Dəyişiklik azdırsa bu da nəticədir: faiz yaxşılaşması həmişə istifadəçi vaxtının yaxşılaşması demək deyil.

11 Rəsmi mənbələr

- Microsoft Learn - Optimize index maintenance
- PostgreSQL - Routine vacuuming
- PostgreSQL - REINDEX

Yoxlama tarixi 28 oktyabr 2026. Nümunələr diaqnostika və tədris üçündür; real bazada heç bir baxım əmri icra edilməyib.

SOLID prinsiplərinin səbəbi və tətbiqi

SOLID obyekt yönümlü dizaynda beş prinsipin baş hərfləridir. Məqsəd proqramın dəyişiklik zamanı hansı hissələrinin təsirlənəcəyini nəzarətdə saxlamaqdır. Prinsiplər sinif sayını artırmaq üçün məcburi qaydalar deyil. Düzgün tətbiq konkret dəyişiklik səbəbini, müqaviləni və asılılığın istiqamətini aydınlaşdırır.

İlkin `files/03_di_architecture.pdf` materialında S, O və D nümunələri var idi, L və I isə qısa verilirdi. Bu dərs bütün beşini ayrıca problem, düzgün model və sərhədlərlə tamamlayır. “Override exception atırsa həmişə LSP pozulub” hökmü də dəqiqləşdirilir.

1 Dizaynı qiymətləndirmək üçün sözlük

Module əlaqəli kod vahididir; sinif, kitabxana və ya paket ola bilər. **Cohesion** bir vahidin içindəki hissələrin eyni məqsədə xidmət etmə dərəcəsidir. **Coupling** vahidlər arasındakı asılılıq dərəcəsidir. Yüksək uyğunluq və idarə olunan bağlılıq dəyişiklik sahəsini anlamağı asanlaşdırır.

Contract, müqavilə, metodun qəbul etdiyi giriş, qaytardığı nəticə və gözlənilən davranışdır. **Client** burada interface-i istifadə edən koddur, mütləq son istifadəçi deyil. **Refactoring** xaricdən müşahidə olunan davranışı qoruyaraq kodun daxili quruluşunu dəyişməkdir. **Abstraction** vacib davranışı göstərib texniki təfərrüatı gizlədən modeldir. Interface onun bir formasıdır.

Sinif məlumat və davranış üçün tipdir. Interface həmin tipin təqdim etdiyi əməliyyat müqaviləsidir. Inheritance, miras alma, bir tipin digər tipdən davranış götürməsidir. Composition, tərkibləmə, bir obyektin başqalarından istifadə etməsidir. Hər “is a”, yəni növ münasibəti, təhlükəsiz davranış əvəzlənməsi demək deyil.

2 S Single Responsibility Principle

SRP, Single Responsibility Principle, vahidin bir əlaqəli dəyişiklik səbəbinə xidmət etməsini istəyir. “Sinifdə yalnız bir metod olmalıdır” demək deyil. Əsas sual budur: bu kodu hansı qaydanın və ya kimin tələbinin dəyişməsi dəyişdirir?

Məsələn, `InvoiceService` hesabın vergi məbləğini hesablayır, SQL ilə saxlayır, PDF düzəldir və email göndərir. Vergi qaydası maliyyə qrupuna, saxlanma texnologiyası infrastruktur qərarına, PDF görünüşü dizayn tələbinə bağlıdır. Bunların hamısı eyni sinfi dəyişdirirsə əlaqəsiz dəyişiklik səbəbləri qarışır.

Mümkün bölgü: `InvoiceCalculator` hesabı hesablayır, `InvoiceStore` saxlayır, `InvoiceRenderer` görünüş verir, `InvoiceSender` göndərir. Bir application service bu işləri istifadə ssenarisinə görə ardıcıllaşdıra bilər. Application service istifadəçinin işi üçün mərhələləri əlaqələndirən xidmət deməkdir. Əlaqələndirmə özü də ayrıca məsuliyyət ola bilər.

Sərhəd: 3 sətirlik sadə funksiyanı 7 sinfə bölmək həmişə yaxşılaşma deyil. Vergi hesablayan sinfin bir neçə köməkçi metodu olması SRP pozuntusu sayılmır. Eyni qaydaya görə birlikdə dəyişən davranışları bir yerdə saxlamaq məntiqlidir. Robert C. Martin-in izahında əlaqəli dəyişiklik səbəbləri və aktorlar vacibdir; aktor kodun davranışına tələb qoyan insan və ya biznes roludur.

3 O Open Closed Principle

OCP, Open Closed Principle, müəyyən dəyişiklik istiqamətində sistemi yeni davranışla genişləndirə bilməyi, sabit əsas məntiqə isə az toxunmağı hədəfləyir. Bütün gələcək dəyişikliklərə qarşı qapalı olmaq mümkün deyil. Hansı dəyişikliklərin tez-tez olacağını seçmək lazımdır.

C#

```
public interface IDiscount
{
    decimal Calculate(decimal total);
}

public sealed class VipDiscount : IDiscount
{
    public decimal Calculate(decimal total) => total * 0.20m;
}

public sealed class Checkout(IDiscount discount)
{
    public decimal Payable(decimal total)
    {
        return total - discount.Calculate(total);
    }
}
```

`decimal` pul hesablaması üçün çox istifadə olunan dəqiq onluq tipdir. `0.20m` həmin tipdə sabitdir. `Checkout` endirim strategiyasını istifadə edir, konkret faiz bilmir. Yeni `StudentDiscount` əlavə edəndə Checkout-u dəyişmədən eyni müqavilə ilə işləyə bilərsən. Composition root-da yeni strategiyanın seçilməsi və qeydiyyatın dəyişməsi normaldır; OCP “heç bir fayl heç vaxt dəyişməsin” demir.

Strategy dəyişən alqoritmi ayrıca müqavilə ilə təqdim edən pattern-dir. Hər switch pis deyil. İki sabit hal üçün sadə switch daha aydın ola bilər. Dəyişiklik tezliyi və variantların müstəqilliyi artanda strategiyaya çıxarmaq faydalıdır. Overengineering hələ olmayan ehtiyac üçün artıq mürəkkəblik yaratmaqdır.

4 L Liskov Substitution Principle

LSP, Liskov Substitution Principle, alt tipin baza tipin yerində istifadə olunanda baza müqaviləsinə güvənən kodu pozmamasını istəyir. Burada tək metod adlarının eyni olması yetmir; davranış qorunmalıdır.

Precondition metod çağırılmazdan əvvəl təmin olunmalı şərtdir. **Postcondition** metod uğurla bitəndə doğru olmalı nəticədir. **Invariant** obyektin etibarlı vəziyyətində həmişə saxladığı qaydadır. Alt tip qəbul edilən girişləri daraltmamalı, vəd edilən nəticəni zəiflətməməli və invariantları pozmamalıdır. Bu, tədris üçün sadələşdirilmiş davranış müqaviləsi çərçivəsidir.

Məsələn, `IFileStore.Write` etibarlı fayl yoluna yazmağı vəd edir. `ReadOnlyStore` həmin müqaviləni qəbul edib həmişə `NotSupportedException` atırsa çağıran kodun gözləntisini pozur. `NotSupportedException` əməliyyat dəstəklənmir xətasıdır. Daha düzgün model `IFileReader` və `IFileWriter`-ı ayırmaqdır; yalnız oxuyan obyekt writer müqaviləsini götürmür.

Ancaq exception atılması təkbaşına LSP pozuntusu deyil. Baza müqaviləsi “disk doludursa yazma xətası qaytara bilər” deyirsə alt tipin bu halda xəta atması gözlənilən davranışdır. Exception proqramın normal axınından çıxan xəta siqnalıdır. Əsas sual yeni alt tipin əvvəl qəbul olunan etibarlı vəziyyət üçün yeni, gözlənilməyən məhdudiyyət gətirib gətirməməsidir.

Klassik dəyişə bilən düzbucaqlı kvadrat nümunəsində baza tipin eni və hündürlüyü müstəqil dəyişə bilməsi müqavilədirsə, kvadratın birini dəyişəndə o birini də dəyişməsi bu gözləntini pozur. Riyazi mənada kvadrat düzbucaqlıdır, amma həmin mutable API müqaviləsi üzrə təhlükəsiz alt tip olmaya bilər. **Mutable** dəyişdirilə bilən deməkdir. Problemi immutable, yaradıldıqdan sonra dəyərləri dəyişməyən model və ya ayrıca fiqur müqaviləsi ilə həll etmək olar.

Müsahibə yoxlaması: yeni implementasiyanı götür, ona bütün köhnə müqavilə testlərini tətbiq et. Məqsəd yalnız “xəta atmır” deyil, giriş, nəticə və vəziyyət qaydalarını eyni saxlamaqdır.

5 I Interface Segregation Principle

ISP, Interface Segregation Principle, client-in istifadə etmədiyi əməliyyatlara məcburi bağlı olmamasını istəyir. Printer, skaner və faks metodları olan böyük `IMachine` sadə printeri lazımsız metodları həyata keçirməyə məcbur edə bilər.

C#

```
public interface IPrinter
{
    void Print(string text);
}

public interface IScanner
{
    byte[] Scan();
}

public sealed class SimplePrinter : IPrinter
{
    public void Print(string text)
    {
        Console.WriteLine(text);
    }
}
```

`byte[]` bayt massivi, yəni ikili məlumat ardıcıllığıdır. `SimplePrinter` scan etməyi vəd etmir. Çoxfunksiyalı cihaz hər iki interface-i həyata keçirə bilər. Bu bölgü real cihazın bacarıqları ilə müqaviləni uyğunlaşdırır.

ISP-ni “hər interface-də bir metod olmalıdır” kimi əzbərləmə. Birlikdə istifadə olunan 4 oxuma əməliyyatını `IOrderReader`-da toplamaq normal ola bilər. Interface sərhədini client-in ehtiyacı və dəyişiklik əlaqəsi müəyyən edir. Lazımsız bölgü də çağırış kodunu çətinləşdirə bilər.

ISP ilə LSP əlaqəsi: oxuya bilməyən writer və ya yaza bilməyən reader üçün nəhəng müqavilə yaratmaq LSP pozuntusuna gətirə bilər. Müqavilələri ehtiyaca görə ayırmaq təhlükəsiz əvəzlənməni asanlaşdırır.

6 D Dependency Inversion Principle

DIP, Dependency Inversion Principle, iş qaydasının SQL, email kitabxanası və HTTP detalları kimi dəyişən mexanizmlərə birbaşa bağlanmasını azaltmaq istəyir. Yüksək səviyyəli siyasət və aşağı səviyyəli detal uyğun abstraksiyaya bağlı olmalıdır. Abstraksiyanın dili isə işin ehtiyacını ifadə etməlidir.

C#

```
public interface IReceiptSender
{
    Task SendAsync(string address, CancellationToken ct);
}

public sealed class CompleteOrder(IReceiptSender sender)
{
    public Task NotifyAsync(string address,
        CancellationToken ct)
    {
        return sender.SendAsync(address, ct);
    }
}
```

Bu fraqmentdə iş xidməti SMTP bağlantısının necə açıldığını bilmir. SMTP email ötürmə protokoludur. `Task` tamamlanmanı təmsil edir, `CancellationToken` ləğv istəyini ötürür. Texniki email adapteri həmin müqaviləni yerinə yetirə bilər. Interface-in özü konkret kitabxananın bütün tiplərini tələb edirsə detal yenə daxilə sızmış ola bilər.

DI bu asılılığı constructor-la vermə üsuludur. DIP dizaynın istiqamətidir. Container-dən istifadə edən, amma konkret SQL sinfinə möhkəm bağlı xidmət DI edir, lakin DIP məqsədinə zəif xidmət edə bilər. Digər tərəfdən container olmadan manual construction ilə DIP-yə uyğun sistem qurmaq mümkündür.

7 Hamısını bir sifariş axınında birləşdirək

`Checkout` sifarişi tamamlasın. Endirim hesablamasını `IDiscount` versin, saxlanmanı `IOrderStore`, bildirişi `IReceiptSender` həyata keçirsin. SRP müxtəlif dəyişiklik səbəblərini ayırır. OCP yeni endirim variantını əlavə etməyə imkan verir. LSP hər endirim implementasiyasının müqaviləni saxlamasını tələb edir. ISP yalnız oxuyan hesabat kodunu yazma metodlarından asılı etmir. DIP iş qaydasını konkret baza və email mexanizmindən ayırır.

Bu bölgü transaction problemini avtomatik həll etmir. Sifariş bazada saxlanıb email göndərilməsi uğursuz ola bilər. Dizayn prinsipi ilə məlumatın çatdırılma zəmanəti ayrı məsələlərdir. Outbox kimi etibarlı hadisə göndərmə yanaşması lazım ola bilər; o, dəyişikliklə hadisəni eyni bazada saxlayıb sonradan göndərir.

8 Müsahibə sualları və məşq

SRP-ni necə tapırsan? Sinfin nə qədər uzun olduğundan əvvəl onun hansı bir-birindən müstəqil tələblərə görə dəyişdiyini soruşuram.

OCP xətanı düzəltməyə mane olurmu? Xeyr. Səhvi düzəltmək üçün mövcud kod dəyişir. Prinsip seçilmiş genişlənmə istiqamətində sabit məntiqə lazımsız toxunmanı azaltmaq üçündür.

Interface varsa DIP təmin olunubmu? Təkbaşına yox. Interface yanlış qatın texniki tipləri ilə doludursa və iş qaydası yenə həmin detala bağlıdırsa asılılıq istiqaməti düzəlməyib.

Məşq. Həm diskə yazan, həm email göndərən, həm endirim hesablayan sinif götür. Dəyişiklik səbəblərini yaz. Sonra endirimi ayır və iki implementasiya qur. Hər ikisinin eyni etibarlı girişlər üçün uyğun nəticə verdiyini yoxla. Sonda qeydiyyat xaricində sifariş axınının dəyişmədən yeni endirim aldığını göstər. Məqsəd sinif sayını deyil, dəyişiklik sahəsini ölçməkdir.

9 Mənbələr

- Robert C. Martin - Single Responsibility Principle
- Liskov və Wing - A Behavioral Notion of Subtyping
- Microsoft Learn - Architectural principles

Yoxlama tarixi 28 oktyabr 2026. Nümunələr müqaviləni göstərən C# fraqmentləridir. Tam biznes və texniki implementasiya ayrıca qurulmalıdır.

Dizayn patternləri və düzgün seçim

Design pattern təkrarlanan dizayn probleminin ümumi həll quruluşudur. Hazır köçürüləcək kod və ya məcburi kitabxana deyil. Pattern seçəndə əvvəl problemi, dəyişən hissəni və qəbul edilən əlavə mürəkkəbliyi izah etmək lazımdır. Bu dərs klassik 23 GoF pattern-in xəritəsini verir, backend-də tez rast gəlinənlərini nümunələrlə açır və əlavə arxitektura pattern-lərini ayrıca ayırır.

İlkin `Design Patterns.pptx` materialında Singleton, Mediator, factory ailəsi, Strategy və Adapter mövcud idi. Abstract Factory-nin “factory-lərin idarəçisi” kimi izahı, DI-nin runtime seçim edə bilmədiyi fikri və mediator-un hər dövrəni həll etdiyi iddiası burada dəqiqləşdirilir. Roadmap sənədi isə dərs planıdır; mövzunun tam izahını əvəz etmir.

1 Pattern dili və üç ailə

GoF, Gang of Four, klassik design patterns kitabının dörd müəllifinə verilən addır: Erich Gamma, Richard Helm, Ralph Johnson və John Vlissides. **Creational** pattern-lər yaradılmanı, **structural** pattern-lər obyektlərin quruluşunu, **behavioral** pattern-lər davranış və ünsiyyəti təşkil edir.

Interface istifadəçiyə verilən müqavilə, **implementation** onun konkret həllidir. **Composition** bir obyektin başqa obyektləri istifadə etməsi, **inheritance** sinfin başqa sinifdən törəməsidir. **Delegation** işi saxlanmış başqa obyektə ötürməkdir. **Client** pattern-dən istifadə edən kod, **context** isə cari davranışı saxlayıb işlədən obyekt ola bilər. Runtime proqramın işlədiyi vaxtdır.

Pattern-in dəyəri adı ilə deyil, dəyişiklik və asılılıq sərhədini necə qurması ilə ölçülür. Sadə bir funksiya problemi aydın həll edirsə əlavə pattern qatları vacib deyil. Eyni kod quruluşu fərqli niyyətlə başqa pattern adını ala bilər; niyyət önəmlidir.

2 Yaradılma patternlərinin xəritəsi

Pattern	Problem və konkret məna
Factory Method	Baza yaradıcı axın müəyyən edir, alt sinif yaratma metodunu əvəz edib hansı məhsulun yaradılacağını seçir. Məsələn, müxtəlif export formatları.
Abstract Factory	Bir-birinə uyğun məhsul ailəsini birlikdə yaradır. Məsələn, eyni mövzuya aid button və checkbox.
Builder	Mürəkkəb obyektin qurulmasını mərhələlərə ayırır. Məsələn, çox seçimli hesabat.
Prototype	Mövcud nümunədən yeni obyekt çıxarır. Kopyalanan daxili obyektlərin paylaşılması ayrıca həll edilməlidir.
Singleton	Seçilmiş kontekstdə bir nüsxəni və ona çıxışı təşkil edir. Paylaşılan vəziyyət və test çətinliyi riski daşıyır.

Product factory-nin yaratdığı obyekt üçün ümumi addır. **Creator** onun yaradılma nöqtəsini təqdim edir. **Clone** obyektin kopyasıdır. **Shallow copy** daxili istinadları da paylaşır; **deep copy** uyğun daxili obyektləri ayrıca kopyalayır. Hansı məlumatın paylaşılması təhlükəsizdirsə kopya qaydası ona görə seçilir.

3 Simple Factory Factory Method və Abstract Factory

Simple Factory bir metodda seçimə görə obyekt yarada bilər. Məsələn, `Create("pdf")` PDF exporter, `Create("csv")` CSV exporter qaytarır. Bu çox işlənən yanaşmadır, amma GoF Factory Method-un inheritance əsasında seçim nöqtəsi ilə eyni anlayış deyil. Hər `Create` adlı metod GoF Factory Method olmur.

C#

```
public interface IExporter
{
    string Export();
}

public abstract class ReportJob
{
    protected abstract IExporter CreateExporter();

    public string Run()
    {
        var exporter = CreateExporter();
        return exporter.Export();
    }
}

public sealed class CsvJob : ReportJob
{
    protected override IExporter CreateExporter()
        => new CsvExporter();
}

public sealed class CsvExporter : IExporter
{
    public string Export() => "Id,Total";
}
```

Bu Factory Method nümunəsində `Run` ümumi axındır, hansı exporter-in yaradılmasını alt sinif seçir. `abstract` baza sinfin həmin davranışı tamamlamadığını, `override` isə alt sinfin onu əvəz etdiyini göstərir. `protected` yalnız sinif və törəmələrinə görünən üzvdür. Nəticə sadələşdirilmiş mətn nümunəsidir, tam CSV fayl yazıcısı deyil. CSV vergüllə ayrılmış cədvəl mətn formatıdır.

Abstract Factory bir-birinə uyğun bir neçə məhsul verir: `IUiFactory.CreateButton()` və `CreateCheckbox()`. `DarkUiFactory` tünd mövzuya uyğun hər iki məhsulu yaradır. Məqsəd factory-ləri idarə etmək deyil, **related product family**, əlaqəli məhsul ailəsinin uyğunluğunu qorumaqdır. Yeni mövzu ailəsi əlavə etmək asandır, amma ailəyə yeni məhsul növü əlavə etmək bütün factory-lərə təsir edə bilər.

DI container yaradılmanı və lifetime-ı təşkil edir. Factory runtime seçimini ayrıca aydın edə bilər. Built-in DI factory delegate və keyed services kimi seçim imkanlarına malikdir. Ona görə “DI varsa factory lazım deyil” və “DI runtime seçim edə bilməz” cümlələrinin ikisi də həddən artıq ümumidir.

4 Builder Prototype və Singleton sərhədləri

Builder çox parametrli obyektin mərhələli qurulmasını idarə edir. Məsələn, `WithTitle`, `AddSection`, `Build`. **Fluent API** metodların obyekti geri qaytarıb zəncir yaratmasıdır. Hər fluent API Builder deyil; Builder-in niyyəti etibarlı mürəkkəb nəticə qurmaqdır. `Build` məcburi hissələrin verildiyini yoxlaya bilər. **Director** varsa qurulma ardıcıllığını idarə edən ayrıca obyektdir, hər builder-də məcburi deyil.

Prototype şablon obyekti kopyalayıb başlanğıc ayarlarını yenidən qurmaq xərcini azalda bilər. Amma sifariş nüsxəsində eyni `List<Item>` istinadını paylaşsan birinin dəyişməsi o birinə təsir edər. Deep copy də database connection və lock kimi resursların mənalı kopyasını avtomatik yaratmır.

Klassik Singleton çox vaxt `static Instance` ilə qlobal çıxış verir. **Static** üzv konkret obyektə deyil tipə bağlıdır. Gizli qlobal mutable state testlərin bir-birinə təsir etməsinə səbəb ola bilər. DI singleton lifetime isə bir provider daxilində xidmət nüsxəsinin paylaşılmasıdır. DI ilə açıq verilən singleton xidməti klassik gizli qlobal çıxışla eyni dizayn deyil. Hər singleton avtomatik SRP pozuntusu sayılmır, amma paylaşılma və yaradılma məsuliyyətləri diqqətlə seçilməlidir.

Birdən çox tətbiq nüsxəsində hərəsinin öz singleton-u olur. Ümumi serverlərarası yeganə lider və ya paylaşılan məlumat lazımdırsa ayrıca koordinasiya tələb olunur. Thread-safe yaradılma obyektin bütün metodlarının thread-safe olması demək deyil. Thread safety paralel çağırışların düzgün nəticəni saxlamasıdır.

5 Struktur patternlərinin xəritəsi

Pattern	Problem və konkret məna
Adapter	Uyğun gəlməyən interfeysi client-in gözlədiyi müqaviləyə çevirir. Köhnə bank API-si üçün yeni ödəniş müqaviləsi nümunədir.
Bridge	Abstraksiya və implementasiyanın ayrıca dəyişməsinə imkan verir. Bildiriş növü ilə göndərmə kanalını ayırmaq olar.
Composite	Tək obyekt və obyekt ağacını eyni müqavilə ilə idarə edir. Fayl və qovluğun ümumi ölçü hesablaması nümunədir.
Decorator	Eyni müqaviləni saxlayan sarğı ilə əlavə davranış verir. Sorğuya log və ya keş əlavə etmək olar.
Facade	Mürəkkəb alt sistemə sadələşdirilmiş giriş təqdim edir. Sifariş üçün bir neçə addımı bir əməliyyatla əlaqələndirir.
Flyweight	Çox obyekt arasında dəyişməz ortaq məlumatı paylaşaraq yaddaş xərcini azaldır. Eyni şrift təsvirini paylaşmaq nümunədir.
Proxy	Başqa obyektə çıxışı nəzarətdə saxlayan əvəzləyici verir. Uzaq xidmət, giriş icazəsi və lazy yaradılma üçün istifadə oluna bilər.

Tree, ağac, valideyn uşaq əlaqəli quruluşdur. Composite-də yarpaq tək elementi, düyün isə başqa elementləri saxlayır. Flyweight-də **intrinsic state** paylaşılabilən daxili məlumat, **extrinsic state** isə çağırışa görə verilən xarici məlumatdır. Məsələn şriftin forması ortaq, hər simvolun ekrandakı yeri xaricidir.

6 Adapter Decorator Proxy fərqi

Adapter-in məqsədi müqaviləni uyğunlaşdırmaqdır. `LegacyBank.PayInCents(int cents)` köhnə metod, `IPayment.Pay(decimal amount)` yeni müqavilədirsə adapter məbləği qəpiyə çevirib köhnə metodu çağırır. Tək tip çevirmə bəs etmir: valyuta, yuvarlaqlaşdırma və xəta mənası da qorunmalıdır. **API** başqa kodun istifadə etdiyi giriş müqaviləsidir.

Decorator eyni müqaviləni saxlayıb davranış əlavə edir. Aşağıdakı nümunə metod çağırışını loglayır. Log işləmə barədə qeyddir.

C#

```
public interface IPrice
{
    decimal Get(int productId);
}

public sealed class LoggedPrice(IPrice inner) : IPrice
{
    public decimal Get(int productId)
    {
        Console.WriteLine($"Price requested: {productId}");
        return inner.Get(productId);
    }
}
```

`inner` sarınan əsas implementasiyadır. `$"...{...}"` string interpolation, yəni ifadənin dəyərini mətnə yerləşdirməkdir. Decorator zənciri ilə ölçmə, log və keş birləşdirilə bilər. Keş əvvəlki nəticəni müəyyən müddət saxlamaqdır. Keş nəticəni qaytarırsa məlumatın yenilənməsi və istifadəçiyə görə ayrılması da müqavilədə düşünülməlidir.

Proxy daha çox obyektə çatmağı idarə edir: məsələn, icazə yoxlayır və ya obyektin uzaqda olmasını gizlədir. Decorator əlavə davranış niyyəti daşıyır. Quruluşları oxşar ola bilər; seçilən ad niyyətə əsaslanır. Facade isə bir neçə alt sistemin istifadəsini sadələşdirə bilər və eyni interface-i saxlamağa məcbur deyil.

7 Davranış patternlərinin xəritəsi

Pattern	Problem və konkret məna
Chain of Responsibility	Sorğunu bir sıra işləyicilərdən keçirir; biri emal edə və ya növbətiyə ötürə bilər. Middleware buna yaxın quruluş nümunəsidir.
Command	Əməliyyat istəyini obyekt kimi saxlayır. Növbələmə, log və uyğun ssenaridə geri alma imkanı verir.
Interpreter	Kiçik dilin qrammatikasını obyektlərlə ifadə edib şərh edir. Sadə qayda dili nümunədir.
Iterator	Kolleksiyanın daxili quruluşunu açmadan elementləri gəzməyə imkan verir. foreach və IEnumerable bu ideyadan istifadə edir.
Mediator	Obyektlərarası qarşılıqlı əlaqəni vasitəçidə əlaqələndirir. Hər obyektin hamısını tanımasını azaldır.
Memento	Obyektin əvvəlki vəziyyətini onun daxilini yaymadan saxlayır və bərpa edir. Redaktorda undo nümunədir.
Observer	Hadisəyə abunə olmuş obyektlərə dəyişiklik bildirişi verir. C# event mexanizmi uyğun nümunədir.
State	Cari vəziyyətə görə davranışı ayrıca state obyektləri ilə dəyişir. Pending və Paid sifariş nümunədir.
Strategy	Eyni məqsəd üçün dəyişən alqoritmləri bir müqavilə altında təqdim edir. Endirim və çatdırılma hesabı nümunədir.
Template Method	Alqoritmin skeletini baza sinifdə saxlayır, bəzi addımları alt sinfə verir. Import axını nümunədir.
Visitor	Sabit obyekt quruluşuna yeni əməliyyatlar əlavə etməyi ayırır. Sintaksis ağacının müxtəlif analizləri nümunədir.

Grammar dil ifadələrinin qaydalarıdır. **Syntax tree** ifadənin hissələrini ağacda göstərən modeldir. **Undo** əvvəlki dəyişiklikdən geri dönməkdir. **Subscription** bildirişə abunəlikdir. Observer-də uzunömürlü publisher, hadisəni yayan obyekt, subscriber-i saxlayıb yaddaşda uzun müddət tuta bilər; abunəliyin bitirilməsi nəzərə alınmalıdır.

Command obyektinin mövcudluğu undo-nu avtomatik mümkün etmir. Xaricə göndərilmiş email və ya ödəniş ayrıca kompensasiya tələb edə bilər. Memento böyük vəziyyətin surətini saxlayırsa yaddaş xərci artır. Visitor yeni əməliyyat əlavə etməyi asanlaşdırır, amma yeni entity tipi əlavə etmək bütün visitor-ləri dəyişməyi tələb edə bilər.

8 Strategy State və Template Method

Strategy-də xarici seçim alqoritmi müəyyən edir: normal, express və ya kuryer çatdırılması. Context seçilmiş hesablamanı çağırır. State-də isə obyektin hazırkı biznes vəziyyəti əməliyyatın mümkünlüyünü və keçidləri müəyyən edir: sifariş Pending-dən Paid-a, Paid-dən Shipped-ə keçir. Shipped vəziyyətdə ödənişi yenidən qəbul etmək məntiqə uyğun olmaya bilər.

Transition, keçid, bir vəziyyətdən başqasına dəyişmədir. Strategy obyektləri adətən eyni məqsədli alternativ alqoritmlərdir. State obyektləri isə vəziyyət keçidlərini və həmin vəziyyətin qaydalarını modelləşdirir. Bu niyyət fərqi oxşar interface quruluşuna baxmayaraq vacibdir.

Template Method baza sinifdə “oxu, yoxla, çevir, saxla” ardıcıllığını sabitləşdirir. Alt sinif CSV və JSON oxuma addımını dəyişə bilər. Bu inheritance-a əsaslanır. Strategy isə davranışı ayrıca obyekt kimi verib composition istifadə edir. Alt siniflər çoxalır və addımlar müstəqil dəyişirsə composition daha çevik ola bilər.

9 Mediator və Observer sərhədləri

Mediator komponentlər arasında əlaqəni idarə edir: bir request uyğun handler-ə yönləndirilə bilər. **Handler** konkret əməliyyatı emal edən koddur. MediatR .NET kitabxanasıdır, Mediator pattern-in tərifi deyil. Kitabxana olmadan da pattern tətbiq edilir.

Mediator-a görə A və B artıq bir-birini constructor-da tanımaya bilər, amma A handler-i B request-i, B handler-i yenə A request-i göndərirsə runtime çağırış dövrəsi qalır. Vasitəçi əlavə etmək məsuliyyət bölgüsünü əvəz etmir. Mediator bütün biznes qaydalarını toplayan həddindən artıq böyük obyektə də çevrilməməlidir.

Observer-də publisher hadisəni elan edir, subscriber-lər reaksiya verir. Mediator isə əlaqənin necə koordinasiya olunacağını bilə bilər. Prosesdaxili event və paylanmış message broker eyni çatdırılma zəmanətinə malik deyil. **Message broker** mesajları proseslər və serverlər arasında daşıyan sistemdir. İnternet bağlantısı, təkrar çatdırılma və davamlı saxlama əlavə problemlərdir.

10 Repository Unit of Work CQRS və Outbox

Bunlar klassik 23 GoF pattern siyahısının hissəsi deyil, amma backend müsahibəsində vacibdir. **Repository** domain obyektlərinə kolleksiya kimi çıxış verərək saxlanma sorğularını bir sərhəddə toplayır. Domain biznes anlayışları və qaydalarıdır. **Unit of Work** bir iş ərzində dəyişən obyektləri izləyib yazmaları əlaqələndirir. EF Core DbContext bu xüsusiyyətlərin bir çoxunu artıq verir. Üzərinə əlavə generic repository yazmaq hər layihədə məcburi deyil.

CQRS, Command Query Responsibility Segregation, dəyişiklik əmrləri ilə oxuma sorğularının model və məsuliyyətlərini ayırır. Eyni bazadan istifadə edə bilər. Ayrı baza, mikroservis və event sourcing məcburi şərt deyil. **Event sourcing** cari vəziyyəti hadisələr tarixindən qurmaq yanaşmasıdır və CQRS ilə eyni anlayış deyil.

Outbox baza dəyişikliyi ilə göndəriləcək hadisəni eyni transaction-da saxlayır. Ayrı worker sonradan hadisəni broker-ə göndərir. Beləcə “sifariş saxlandı, amma hadisə itdi” pəncərəsi daralır. Ancaq hadisə təkrar göndərilə bilər. **Idempotency** eyni əməliyyat təkrar gələndə əlavə yanlış təsir yaratmamaqdır; məsələn, eyni hadisə ID-si ilə iki dəfə kredit yazmamaq. Sadə “SaveChanges et, sonra Publish et” iki müstəqil sistem arasında atomiklik yaratmır.

Retry keçici uğursuzluqda təkrar cəhddir. **Timeout** gözləmə müddətini məhdudlaşdırır. **Circuit breaker** davamlı xəta alan asılılığa çağırışları müvəqqəti dayandırır. **Bulkhead** bir hissənin bütün resursları tutmaması üçün paralel işi məhdudlaşdırır. Bunlar GoF ailəsindən ayrı əməliyyat və etibarlılıq pattern-ləridir. Ödənişi kor retry etmək iki dəfə tutmaya səbəb ola bilər; idempotency ilə birlikdə düşünülməlidir.

11 Seçim sualları və məşq

Ödəniş kitabxanasının metodu uyğun gəlmir? Adapter. Müqavilə və xəta mənası çevrilir.

Eyni metoda keş əlavə etmək istəyirsən? Decorator uyğun ola bilər. Müddət, istifadəçi sərhədi və invalidation, yəni köhnə keşin silinməsi, müəyyən edilir.

Alqoritmi dəyişmək istəyirsən? Strategy. Obyektin biznes vəziyyəti davranışı dəyişirsə State-i düşün.

Bir-birinə uyğun bir neçə məhsul yaranmalıdır? Abstract Factory. Bir create metodunda seçim etmək kifayətdirsə Simple Factory də yetə bilər.

Məşq. Çatdırılma qiyməti üçün iki Strategy yaz. Xarici kuryer API-sini Adapter ilə həmin müqaviləyə uyğunlaşdır. Çağırış sayını Decorator ilə qeyd et. Sonra hər pattern-i çıxarsan nə çətinləşəcəyini yaz. Cavab “pattern adı azaldı” deyil, konkret dəyişiklik və müqavilə ilə izah olunmalıdır.

12 Mənbələr və əlavə oxu

- Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides - Design Patterns kitabı. Klassik 23 pattern kataloqu üçün əsas əsərdir; burada kitabdan uzun sitat istifadə edilmir.
- Martin Fowler - Repository
- Martin Fowler - Unit of Work
- Martin Fowler - CQRS
- Microsoft Azure Architecture Center - Cloud design patterns

Veb mənbələr 28 oktyabr 2026 tarixində yoxlanılıb. C# nümunələri pattern-in əlaqələrini göstərir, real ödəniş və mesaj göndərmə əməliyyatı icra etmir.

Task async await və paralel işin məntiqi

`Task` əməliyyatın tamamlanmasını və nəticəsini təmsil edir. `async` metodu asinxron gözləmələr yazmağa hazırlayır. `await` isə uyğun əməliyyatın tamamlanmasını cari thread-i bloklamadan gözləməyə imkan verir. Bu üç anlayışdan heç biri təkbaşına “mütləq yeni thread açılır” demək deyil.

İlkin `Sinxron Proqramlaşdırma.pdf` və `files/01_dotnet_core_fundamentals.pdf` materiallarında async-in ümumi izahı var idi. Task-ın həmişə ThreadPool üzərində işləməsi, await-dən sonra mütləq eyni thread-ə qayıtması və ValueTask-ın həmişə stack-də olması kimi ümumiləşdirmələr burada düzəldilir. Cancellation, exception, məhdud paralellik və context ömrü də əlavə olunur.

1 Proses thread concurrency parallelism

Process işləyən proqramın əməliyyat sistemi mühitidir. **Thread** onun kod icra edən axınıdır. **CPU core** prosessorun hesablama aparan nüvəsidir. **ThreadPool** .NET-in qısa işlər üçün idarə etdiyi worker thread-lər mexanizmidir. Hər iş üçün sıfırdan thread yaratmaq əvəzinə mövcud resurslardan istifadə edir.

Concurrency, eyni vaxtlılıq, bir neçə işin vaxtları üst-üstə düşən şəkildə irəliləməsidir. Bir nüvədə növbələşmə və I/O gözləməsi ilə də mümkündür. **Parallelism**, paralellik, müxtəlif iş hissələrinin həqiqətən eyni anda icrasıdır. **Asynchrony**, asinxronluq, işin tamamlanmasını gözləyərkən çağıran icra axınının bloklanmamasına imkan verən modeldir. Bu anlayışlar əlaqəlidir, amma sinonim deyil.

I/O, Input Output, disk, şəbəkə və başqa xarici mənbələrlə məlumat mübadiləsidir. **I/O-bound** işdə əsas vaxt cavabı gözləməyə gedir. **CPU-bound** işdə əsas vaxt hesablamaya sərf olunur. Şəkli hesablamaq CPU, şəkli serverdən yükləmək I/O üstünlüklü ola bilər. Real əməliyyat hər iki mərhələdən ibarət ola bilər.

Latency bir əməliyyatın cavab müddətidir. **Throughput** müəyyən vaxtda tamamlanan iş sayıdır. Async serverdə eyni gözləmə müddətində daha çox sorğuya xidmət etməyə kömək edə bilər; bir uzaq serverin 500 ms cavabını avtomatik 50 ms etmir.

2 Task əməliyyatın vədidir thread deyil

`Task` nəticə dəyəri olmayan tamamlanmanı, `Task<T>` isə T tipli nəticəsi olan tamamlanmanı təmsil edir. Məsələn, `Task<int>` sonradan bir tam ədəd verəcək əməliyyatdır. Task nəticələnə bilər, xəta ilə bitə bilər və ya ləğv oluna bilər. **Faulted** xəta ilə bitmiş, **Canceled** ləğv olunmuş, **RanToCompletion** uğurla tamamlanmış vəziyyətdir.

`Task.Run` CPU işini adətən ThreadPool-da planlayır. Amma `HttpClient.GetStringAsync` ilə şəbəkə cavabı gözlənərkən həmin Task üçün ayrıca thread-in bütün vaxt boyu tutulması tələb olunmur. Əməliyyat sistemi I/O tamamlanma bildirişini verəndə kodun davamı planlaşdırıla bilər. `Task.CompletedTask` artıq bitmiş Task-dır; ona xüsusi işçi thread lazım deyil.

Scheduler işin harada və nə vaxt icra ediləcəyini təşkil edən mexanizmdir. **Continuation** gözləmədən sonra işləyəcək davam kodudur. Task və continuation-ların sayı thread sayına bərabər deyil. 1000 açıq HTTP gözləməsi avtomatik 1000 işçi thread demək deyil.

3 Async metod ilk haraya qədər işləyir

C#

```
static async Task DemoAsync()
{
    Console.WriteLine("A");
    await Task.Delay(1000);
    Console.WriteLine("B");
}

Task work = DemoAsync();
Console.WriteLine("C");
await work;
```

`DemoAsync` çağırılan thread-də başlayır. A yazılır. Delay hələ tamamlanmadığı üçün metod gözləmə nöqtəsində çağırana qayıdır. C yazılır. Gecikmə tamamlanandan sonra B yazılır. Bu nümunədə gözlənən sıra A, C, B-dir. `Task.Delay` timer, vaxt bildirişi, ilə tamamlanan gözləmədir; `Thread.Sleep` kimi thread-i həmin müddətdə bloklamır.

Await olunan əməliyyat artıq bitibsə metod dərhal davam edə bilər və çağırana gözləmə üçün qayıtmaz. Ona görə “hər await thread-i buraxır və dəyişir” cümləsi doğru deyil. Async metodun await-ə qədər olan ağır CPU dövrü də çağıran thread-də işləyir. Tək `async` sözünü əlavə etməklə CPU işi arxa plana keçmir.

State machine metodun hansı gözləmə mərhələsində olduğunu və lazım olan lokal dəyərləri saxlayan vəziyyət mexanizmidir. Kompilyator async metodu buna uyğun çevirir. Daxili state rəqəmləri compiler təfərrüatıdır; “0 həmişə başlanğıcdır, -1 həmişə bitdi” kimi universal cədvəl əzbərləmək lazım deyil. Kompilyator C# mətnini işləyən formaya çevirən alətdir.

4 I/O və CPU üçün fərqli seçim

C#

```
// I/O ucun hazir async API-ni birbasa gozle.
string text = await http.GetStringAsync(url, ct);

// UI-de agir CPU isini ayrica planlamaq olar.
int value = await Task.Run(() => Calculate(), ct);
```

`http` HttpClient, `url` şəbəkə ünvanı, `ct` cancellation token-dir. `Calculate` sənin hesablama metodundur. Lambda `() => Calculate()` sonra icra ediləcək işi təqdim edir. **UI**, User Interface, istifadəçi interfeysidir. UI thread-i düymə və görünüş yeniləmələrini idarə edir; uzun CPU işi onu dondura bilər.

ASP.NET Core-da sadəcə bir sinxron baza çağırışını Task.Run ilə sarımaq onun I/O-sunu həqiqi asinxron etmir; ThreadPool thread-i yenə gözləyə bilər. Hazır async baza API-sini istifadə et. Serverdə ağır CPU işini əlavə Task.Run-a ötürmək ümumi hesablama gücünü artırmır. İşin həcmi böyükdürsə ayrıca worker, növbə və ya uyğun resurs planlaması lazım ola bilər.

`Task.Run(..., ct)` token-i iş hələ başlamayıbsa planlanmanı ləğv edə bilər. İşləyən `Calculate` token-i yoxlamırsa onu zorla dayandırmır. **Cooperative cancellation** əməliyyatın ləğv siqnalını özü nəzərə almasıdır.

5 Ardıcıl await və birlikdə başlatma

C#

```
// Ardicil: ikinci is birinci bitenden sonra baslayir.
var a = await GetPriceAsync(ct);
var b = await GetStockAsync(ct);

// Musteqil isler: evvel her ikisini baslat.
var priceTask = GetPriceAsync(ct);
var stockTask = GetStockAsync(ct);
await Task.WhenAll(priceTask, stockTask);

var price = await priceTask;
var stock = await stockTask;
```

`WhenAll` verilmiş bütün Task-lar tamamlananda tamamlanan Task qaytarır. Onları avtomatik yaratmır və mütləq ayrı CPU nüvələrində işlətdiyi mənasına gəlmir. Yuxarıda əməliyyatları metod çağırışları başlayır. İşlər bir-birindən müstəqil olmalıdır. İki 300 ms və 500 ms xarici gözləmə üçün ideal sadə modeldə ardıcıl vaxt 800 ms, üst-üstə düşən vaxt təxminən 500 ms ola bilər; real servis, şəbəkə və limitlər nəticəni dəyişir.

Eyni DbContext ilə iki paralel sorğunu WhenAll-a vermək düzgün deyil. DbContext eyni anda bir neçə əməliyyatı təhlükəsiz icra etmək üçün nəzərdə tutulmayıb. Ayrı context-lər istifadə edəndə də connection pool və baza resurslarını tükətməmək lazımdır. Connection pool əvvəl açılmış fiziki bağlantıları təkrar istifadə edən mexanizmdir.

6 WhenAll xətası və WhenAny

WhenAll bütün verilmiş işlərin bitməsini gözləyir. Biri xəta atanda digərlərini avtomatik dayandırmır. Bir və ya daha çox fault varsa ümumi Task faulted olur. Fault yox, amma cancellation varsa canceled ola bilər. `await` çox vaxt bir exception-u yenidən atır; bütün xətalar ümumi Task-ın `Exception` məlumatında saxlanır.

C#

```
Task all = Task.WhenAll(tasks);
try
{
    await all;
}
catch
{
    if (all.Exception is not null)
        foreach (var error in all.Exception.InnerExceptions)
            Console.WriteLine(error.Message);
    throw;
}
```

`tasks` əvvəl yaradılmış Task kolleksiyasıdır. `Exception` xəta obyektidir. `InnerExceptions` toplanmış xətaları verir. `throw;` ilkin xəta izini qoruyaraq xətanı yuxarı ötürür. Cancellation da catch-ə düşə bilər, bu halda `Exception` null ola bilər; buna görə yoxlama var.

`WhenAny` ilk tamamlanan Task-ı verir. “Tamamlandı” uğur, fault və ya cancellation ola bilər. Qalib Task-ı ayrıca await etmək onun həqiqi nəticəsini və ya xətasını almaq üçündür. Qalan Task-lar avtomatik ləğv olunmur.

`WaitAsync(timeout)` kimi gözləməyə vaxt məhdudiyyəti qoymaq əməliyyatın özünü mütləq dayandırmır. Əsas əməliyyat cancellation dəstəkləyirsə token ona da ötürülməlidir. Timeout gözləmənin bitməsi, cancellation isə əməliyyata dayanma istəyidir; iki ayrı müqavilədir.

7 CancellationToken və vaxt sərhədi

`CancellationToken` yalnız siqnalı oxumaq üçündür. `CancellationTokenSource` siqnalı yaradır və ləğv edir. `CancelAfter` müəyyən müddətdən sonra ləğv istəyir. `ThrowIfCancellationRequested` istək varsa `OperationCanceledException` atır; bu, əməkdaşlıq edən kodda normal ləğv yoludur.

C#

```
using var timeout = new CancellationTokenSource();
timeout.CancelAfter(TimeSpan.FromSeconds(5));

using var linked = CancellationTokenSource
    .CreateLinkedTokenSource(userToken, timeout.Token);

string text = await http.GetStringAsync(url, linked.Token);
```

Linked token hər iki mənbədən biri ləğv istəyəndə siqnal verir. `userToken` istifadəçinin və ya HTTP sorğusunun token-i ola bilər. `TimeSpan` vaxt müddətidir. `using` source-un resurslarını iş bitəndə təmizləyir. Token-i aşağıdakı async metodlara ötürməsən siqnal zəncirin sonunda itə bilər. Baza sürücüsünün cancellation-u qəbul etməsi serverdə hər şeyin dərhal geri alındığına zəmanət vermir; əməliyyatın nəticəsi ayrıca nəzərə alınmalıdır.

8 Result Wait və deadlock

`.Result` və `.Wait()` Task bitənə qədər çağıran thread-i bloklaya bilər. **Deadlock** tərəflərin bir-birini gözləyib irəliləyə bilməməsidir. UI-də metod `.Result` ilə bloklanır, həmin metodun await-dən sonrakı davamı UI thread-də işləmək istəyirsə dairə yaranır: UI nəticəni, nəticə UI-ni gözləyir.

SynchronizationContext davamın hansı mühitdə icra olunacağını bildirən abstraksiyadır. UI mühitində bu çox vaxt UI thread-idir. ASP.NET Core-da standart olaraq klassik request SynchronizationContext yoxdur. Buna görə UI-dəki həmin konkret deadlock mexanizmi eyni formada gözlənilmir, amma blocking yenə ThreadPool starvation yarada bilər. **Starvation** işləməyə hazır işlərin thread tapmaqda uzun gözləməsidir. Digər kilid deadlock-ları hər mühitdə mümkündür.

`ConfigureAwait(false)` həmin await üçün uyğun context-i tutub davamı ora qaytarmaq tələbini azaldır. “Mütləq yeni thread-də işlə”, “bütün deadlock-ları həll et” və “ExecutionContext-i sil” demək deyil. **ExecutionContext** AsyncLocal kimi məntiqi çağırış məlumatlarının axın mexanizmidir. Kitabxana kodunda context tələb olunmursa faydalı ola bilər; UI görünüşünü dəyişən kodda thread tələbi qorunmalıdır. ASP.NET Core tətbiq kodunda onu hər sətirə yazmaq universal məcburiyyət deyil.

9 Async void və resurs ömrü

`async void` çağırana await edilən Task vermir. Ona görə tamamlanmanı və xətanı adi Task kimi izləmək olmur. Əsas istisna void imzalı event handler-dir: düymə hadisəsi kimi. Digər iş metodları üçün `Task` və ya `Task<T>` daha uyğun müqavilədir.

Fire and forget işi başlayıb tamamlanmasını izləməməkdir. HTTP sorğusunda `_ = SendAsync()` yazmaq həmin işin etibarlı tamamlanmasına zəmanət vermir. Sorğu scope-u bağlana, istifadə olunan DbContext dispose oluna və xəta müşahidəsiz qala bilər. Etibarlı fon işi üçün host-un idarə etdiyi worker, öz scope-u və davamlı iş tələb olunursa növbə lazımdır.

C#

```
// Resurs await tamamlanana qeder aciq qalir.
static async Task<string> ReadAsync(string path)
{
    using var reader = new StreamReader(path);
    return await reader.ReadToEndAsync();
}
```

`StreamReader` fayldan mətn oxuyur. `using` reader-i metoddan çıxanda təmizləyir. Bu nümunədə await əhəmiyyətlidir: async işi bitmədən scope-dan çıxıb resursu bağlamaq olmaz. **Dispose** fayl kimi resursları buraxır; managed yaddaşı dərhal silmir.

10 ValueTask nə vaxt lazımdır

`ValueTask<T>` nəticəni birbaşa, Task vasitəsilə və ya xüsusi mənbə ilə təmsil edə bilən value type-dır. Tez-tez sinxron tamamlanan yüksək tezlikli əməliyyatlarda bəzi Task allocation-larını azalda bilər. **Allocation** yeni obyekt üçün yaddaş ayırmaqdır. Task üçün hazır cached, yəni təkrar istifadə olunan tamamlanmış nəticələr də ola bilər; “hər Task çağırışı mütləq yeni heap obyekti yaradır” doğru deyil.

ValueTask value type olsa da “həmişə stack-dədir” deyil. Sinfin sahəsində və async state machine daxilində saxlanması kimi hallar var. **Stack** icra çağırışları üçün yaddaş sahəsi, **heap** obyektlərin idarə olunan yaddaş sahəsidir; saxlanma yeri təkcə tipin adından müəyyən edilmir.

Ümumi API-lərdə başlanğıc seçim Task-dır. ValueTask seçmək istehlakçı qaydalarını artırır: ümumi qayda kimi bir dəfə await et, saxlayıb dəfələrlə gözləmə. Bir neçə dəfə gözləmək lazımdırsa bir dəfə `AsTask()` ilə Task-a çevirib onu istifadə et. Dəqiq məhdudiyyət mənbə implementasiyasından asılıdır; təhlükəsiz ümumi müqaviləyə əməl etmək lazımdır. Ölçmə olmadan “ValueTask daha yenidir, deməli hər yerdə yaxşıdır” nəticəsi çıxarma.

11 Race condition və məhdud paralellik

Race condition nəticənin əməliyyatların nəzarətsiz gəliş sırasından asılı olmasıdır. `counter++` oxu, artır, yaz addımlarından ibarət olduğundan iki thread eyni ilkin dəyəri oxuyub bir artımı itirə bilər. `Interlocked.Increment` bir artımı digər paralel artımlara qarşı bölünməz edir. Bu **atomic operation** anlayışıdır; baza transaction-unun bütün biznes işi üçün atomikliyi ilə eyni ölçüdə deyil.

Lock kritik hissəyə eyni anda məhdud girişi təşkil edir. Adi `lock` bloku içində await istifadə edilmir. Asinxron gözləmə ilə giriş məhdudlaşdırmaq üçün `SemaphoreSlim.WaitAsync` uyğun ola bilər. Semaphore müəyyən sayda paralel icazə verir. `Release` adətən `finally` daxilində çağırılır ki, xəta olanda da icazə geri qaytarılsın.

C#

```
await Parallel.ForEachAsync(
    ids,
    new ParallelOptions
    {
        MaxDegreeOfParallelism = 4,
        CancellationToken = ct
    },
    async (id, token) =>
    {
        await FetchAsync(id, token);
    });
```

`ids` işlənəcək identifikatorlar, `FetchAsync` sənin I/O metodundur. Maksimum 4 paralel iş resurs yükünü məhdudlaşdırır. Bu rəqəm sadəcə nümunədir; servis limitinə və connection pool-a görə ölçülməlidir. `Parallel.ForEach`-ə `async` lambda vermək async void tipli səhvlərə səbəb ola bilər; asinxron iş üçün uyğun API və müqavilə seç.

12 Müsahibə cavabları və məşq

Await hansı thread-də davam edir? Mühit, context, scheduler və əməliyyatın artıq tamamlanıb tamamlanmamasından asılıdır. Eyni thread-ə zəmanət yoxdur; həmişə başqa thread-ə keçid də yoxdur.

Async proqramı sürətləndirirmi? I/O gözləməsində thread resursunu sərbəst saxlaya bilər və müstəqil işləri üst-üstə salmaq olar. CPU işini avtomatik sürətləndirmir. Daha çox sorğuya xidmətlə bir sorğunun cavab vaxtını ayırmaq lazımdır.

Məşq. İki müstəqil `Task.Delay`, biri 300 ms, o biri 500 ms olsun. Əvvəl ardıcıl await et, sonra əvvəl hər ikisini çağırıb WhenAll gözlə. Vaxtı `Stopwatch` ilə ölç; o, keçən vaxtı ölçən .NET sinfidir. Sonra bir işdə xəta yarat: digərinin avtomatik dayanmadığını gör. Nəhayət shared sayğacı paralel artırıb adi ++ ilə Interlocked fərqini müşahidə et. Vaxt rəqəmləri yükə görə dəyişəcək.

13 Rəsmi mənbələr

- Microsoft Learn - Asynchronous programming
- Microsoft Learn - Async scenarios
- Microsoft Learn - Task based Asynchronous Pattern
- Microsoft Learn - Task.WhenAll
- Stephen Toub - ConfigureAwait FAQ
- Stephen Toub - Understanding ValueTask

Yoxlama tarixi 28 oktyabr 2026. C# nümunələri tədris fraqmentləridir; şəbəkə və baza çağırışları real servisdə icra edilməyib.